SW개발/HW제작 설계서

프로젝트 명 : 공공데이터 AI 데이터셋 자동 표준화 및 검증 플랫폼 개발

2026. 09. 08 작성 · 2026. 10. 12 개정

(26_HC277)

수행 단계별 주요 산출물

단계	산출물	일반 (모바일 APP·Web 등)	응용 소프트웨어 (빅데이터·인공지능 등)	응용 하드웨어 (IoT·로봇 등)	본 설계서
환경 분석	시장/기술 환경 분석서	△	△	△	수록
	설문조사 결과서	△	△	△	-
	인터뷰 결과서	△	△	△	-
요구사항 분석	요구사항 정의서	○	○	○	수록
	유즈케이스 정의서	△	△	△	-
아키텍처 설계	서비스 구성도(시스템 구성도)	○	○	○	수록
	서비스 흐름도(데이터 흐름도)	△	○	△	수록
	UI/UX 정의서	△	△	△	-
	하드웨어/센서 구성도	-	-	○	해당 없음
기능 설계	메뉴 구성도	○	○	○	수록
	화면 설계서	○	○	△	수록
	엔티티 관계도	○	○	△	수록
	기능 처리도(기능 흐름도)	○	○	○	수록
	알고리즘 명세서/설명서	△	○	○	수록
	데이터 수집처리 정의서	-	○	-	수록
	하드웨어 설계도	-	-	○	해당 없음
개발 / 구현	프로그램 목록	○	○	○	수록
	테이블 정의서	○	○	△	수록
	핵심 소스코드	○	○	○	수록

- ※ ○ 필수, △ 선택
- 본 프로젝트는 응용 소프트웨어(빅데이터·인공지능) 유형에 해당하며, 필수 산출물 전체와 시장/기술 환경 분석서를 수록함
- 전용 하드웨어를 쓰지 않으므로 하드웨어 관련 산출물은 작성하지 않음

시장/기술 동향 분석

1. 시장 동향

- AI 학습 데이터 시장은 2024년 약 26억 달러에서 2030년 86억 달러로 연평균 21.9% 성장 전망 (Grand View Research, 2025)
- 생성형 AI·LLM 확산으로 출처와 품질이 확인된 학습·분석용 데이터 수요 증가
- 국내 공공데이터는 OECD 공공데이터 평가(OURdata Index)에서 4회 연속 1위 기록, 개방 건수는 꾸준히 증가
- 개방은 원천 API 제공까지이고, 분석·학습에 바로 쓸 수 있는 표 형태로 가공하는 작업은 이용자 몫으로 남아 있음
- → 공공데이터를 수집·정제·검증해 재사용 가능한 테이블과 데이터셋으로 만들어 주는 도구가 필요한 시점

2. 기술 성숙도 단계

단계	정의	개요
Level 0	수동 처리	개별 스크립트로 API를 호출하고 수기로 정제. 표준·재현성 없음
Level 1	부분 자동화	수집·정제를 스크립트화. 형식 변환 일부만 자동화
Level 2	파이프라인화	수집→정제→검증을 ETL 파이프라인으로 연결 (dbt·Airflow 등)
Level 3	**선언적·재현 빌드**	**명세(BuildSpec) 기반 실행, 출처·Manifest 기록, 스냅샷 관리, 다중 포맷 내보내기 ★ 본 프로젝트**
Level 4	지능형 자동화	스키마 자동 매핑, 품질 자동 개선, LLM 기반 데이터 큐레이션 (지향점)

시장/기술 동향 분석

3. 경쟁 제품 비교

기능	본 프로젝트	공공데이터 포털	HF Datasets	Kaggle	dbt·ETL	DVC
공공 API 직접 수집	○	○	×	×	△	×
자동 정규화·표준화	○	×	×	×	○	×
선언적 빌드(명세)	○	×	×	×	○	△
다중 포맷 내보내기	○	△	○	△	×	×
데이터셋 카드 자동 생성	○	×	△	△	×	×
재현성(Manifest·버전)	○	×	△	×	△	○
허브 배포(HF·Kaggle)	○	×	○	○	×	×
비개발자용 웹 UI	○	△	△	○	×	×
국내 공공데이터 특화	○	○	×	×	×	×

- ○ 제공, △ 일부 제공, × 미제공

4. 차별화 비교

구분	기존 접근	본 프로젝트
공공데이터 포털	원천 API 제공까지. 분석·학습 적합성은 이용자가 확인	정제·검증·분할·문서화를 거친 테이블과 데이터셋으로 산출
범용 ETL 도구 (Airflow 등)	파이프라인 실행은 되나 도메인 지식과 검증 규칙은 사용자 몫	kpubdata가 제공하는 공공 API 접근 위에 스키마 계약·품질 규칙을 명세로 선언
데이터 허브 (Hugging Face 등)	배포·공유 계층. 생성 과정은 남지 않음	생성 규칙·실행 기록·입력 지문을 Manifest로 남기고 카드와 함께 게시
일반 LLM 데이터 도구	LLM이 스키마와 내용을 추정해 생성	LLM 답변을 Builder 근거와 대조하고 사용자 승인 전에는 실행하지 않음

요구사항 정의서

요구사항 ID	요구사항명	기능ID	기능명	세부사항	예외사항	상태
A01	데이터 수집	A01_B01	공공 API 수집	kpubdata로 공공 API를 호출해 원천 레코드를 페이지 단위로 수집·저장	수집 실패 시 해당 소스를 실패로 기록하고 나머지 소스는 계속 처리	구현
		A01_B02	파일·URL 수집	업로드 파일(CSV·JSON·JSONL·Parquet)과 HTTPS URL(JSON·JSONL·CSV)을 소스로 사용	허용되지 않은 형식·주소는 명세 검증에서 거부	구현
		A01_B03	소스 카탈로그	kpubdata에 등록된 제공 기관·데이터셋 목록을 조회	조회 실패 시 오류 상태와 다시 시도 제공	구현
		A01_B04	정기 수집	GitHub Actions 일정에 따라 분야별 데이터를 주기적으로 수집·게시	실패 시 이슈를 자동 등록	구현
A02	데이터 가공·품질	A02_B01	BuildSpec 작성	소스·스키마 계약·품질·분할·내보내기를 하나의 명세로 선언	필수 항목 누락 시 실행 전에 오류 목록 반환	구현
		A02_B02	BuildSpec 검증	필수 항목, 소스 종류, 스키마 계약, 분할, 라이선스 등 15개 영역 검사	위반 사항을 code·path·message로 모두 모아 반환	구현
		A02_B03	결과 미리보기	저장 없이 소스별 스키마와 샘플 행을 확인	미리보기 실패 시 원인을 화면에 표시	구현
		A02_B04	Bronze 원천 보존	수집한 원천 레코드와 수집 시각·파라미터·건수·체크섬을 보존	저장 실패 시 해당 소스를 실패 처리	구현
		A02_B05	Silver 정규화	DuckDB로 결측 토큰·컬럼 병합·이름 변경·자리 채움·형 변환·파생 컬럼을 순서대로 적용	형 변환으로 값이 사라지면 오류 처리	구현
		A02_B06	품질 검사	필수 컬럼, 자료형, 중복률, 결측률, 최소 행 수, 값 범위, 컬럼 간 비교 검사	fail은 Gold 전에 중단, warn은 기록 후 진행	구현
		A02_B07	Gold 선택·분할	행 필터와 컬럼 선택, 비율·키 기준 분할, 두 소스 조인	분할 비율 합이 맞지 않으면 검증 오류	구현

요구사항 정의서

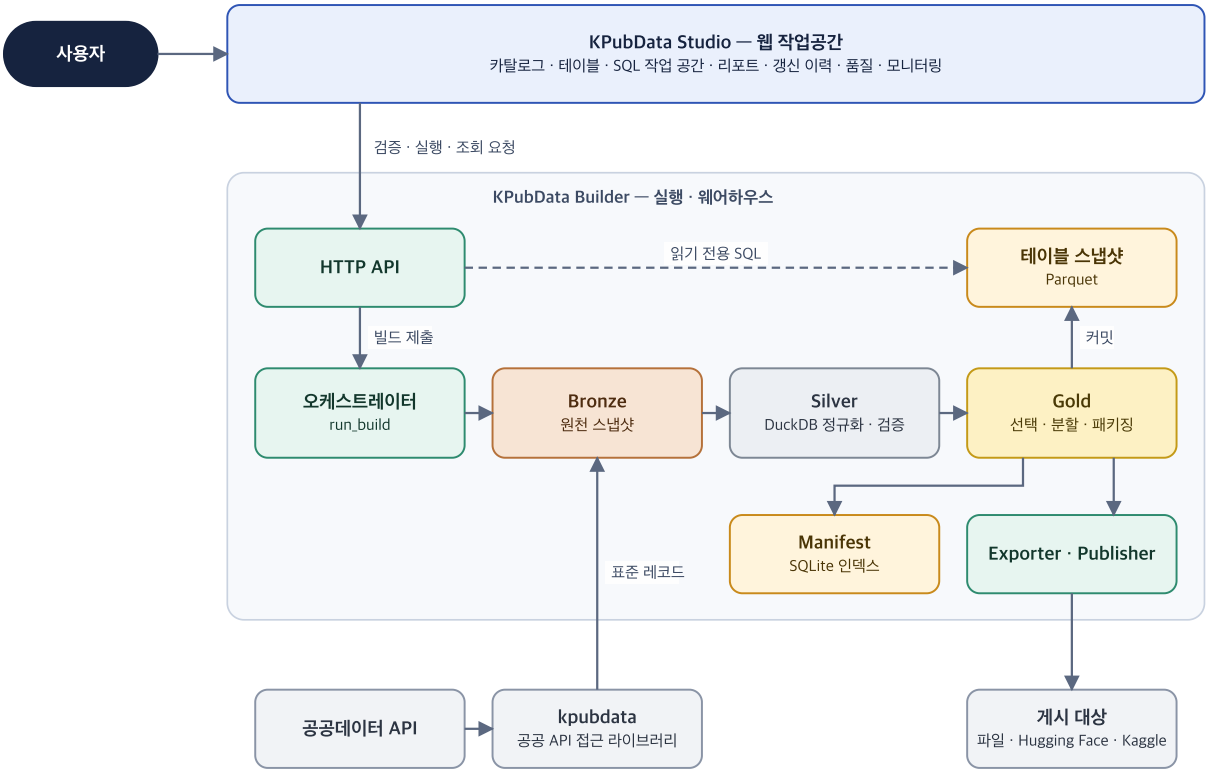
요구사항ID	요구사항명	기능ID	기능명	세부사항	예외사항	상태
A03	산출물 생성·배포	A03_B01	데이터셋 카드	스키마·샘플·출처·처리 내역·개인정보 항목을 담은 README.md와 card.json 생성	빈 항목이 있으면 게시를 막음	구현
		A03_B02	다중 포맷 내보내기	CSV·JSONL·Parquet·Markdown·Hugging Face·Kaggle 형식으로 출력	등록되지 않은 형식은 검증에서 거부	구현
		A03_B03	Manifest 기록	상태·입력·산출물·행 수·출처·품질 결과·실패 내역을 manifest.json으로 기록	Manifest 없이 끝난 실행은 종료 기록으로 따로 남김	구현
		A03_B04	테이블 스냅샷	성공한 실행의 Gold를 웨어하우스 스냅샷으로 커밋하고 최근 3개 유지	동시 갱신 충돌 시 커밋을 거부하고 사유 기록	구현
		A03_B05	외부 게시	산출물을 로컬·Hugging Face·Kaggle로 게시. 준비 상태 점검 후 실행	라이선스·개인정보 검사 미통과 시 게시 차단	구현
A04	웹 작업공간(Studio)	A04_B01	홈	조치가 필요한 테이블, 최근 스냅샷·분석, 연결 문제 표시	조회 실패 시 영역별 오류 표시	구현
		A04_B02	카탈로그·테이블 만들기	소스를 찾아 3단계 마법사로 테이블 생성	검증 실패 시 실행 차단	구현
		A04_B03	테이블·테이블 상세	스냅샷·스키마·프로파일·미리보기·품질 확인	웨어하우스가 없으면 실행 기준 보기로 전환	구현
		A04_B04	SQL 작업 공간	스냅샷에 읽기 전용 SQL 실행, 분석 저장, 집계 차트, 결과 내보내기	허용되지 않은 SQL은 Builder가 거부	구현
		A04_B05	리포트	실행 근거를 바탕으로 리포트 작성, Markdown·HTML 내려받기	브라우저에만 저장됨을 화면에 표시	구현
		A04_B06	갱신 이력·게시	실행 목록·상세·산출물 확인, 재실행, Hugging Face 게시	실행 실패 시 실패 단계와 코드 표시	구현
		A04_B07	연결·설정	제공 기관 자격 증명 관리, Builder 연결 상태·버전 확인	최소 계약 버전 미만이면 경고 배너	구현

요구사항 정의서

요구사항 ID	요구사항명	기능ID	기능명	세부사항	예외사항	상태
A05	운영 자동화	A05_B01	CI	Builder: ruff·mypy·pytest(Python 3.10~3.13). Studio: ESLint·Vitest·Playwright	검사 실패 시 병합 차단	구현
		A05_B02	계약 호환 점검	Builder API 계약과 Studio 스키마의 불일치를 CI에서 검사	불일치 시 실패	구현
A06	데이터 보안·거버넌스	A06_B01	개인정보 검사·마스킹	주민등록번호·전화번호·이메일·사업자번호 패턴과 컬럼명 규칙으로 검사, 선언한 컬럼 마스킹	block 모드에서 검출 시 해당 소스 중단	구현
		A06_B02	라이선스·재배포 판정	소스별 재배포 조건(허용·비상업·금지·미확인) 중 가장 엄격한 값 적용	게시할 때 라이선스 미기재면 검증 오류	구현
		A06_B03	읽기 전용 SQL 검사	단일 SELECT만 허용, 테이블 함수·비결정 함수·외부 접근 차단	위반 시 실행하지 않고 거부	구현
		A06_B04	인증·자격 증명	OIDC(Keycloak) 로그인, 제공 기관 키 암호화 저장	토큰 갱신 실패 시 로그인 화면으로 이동	구현
A07	AI 보조	A07_B01	Ask KPubData	화면 문맥과 Builder 근거를 바탕으로 답변·SQL·다음 동작 제안	근거에 없는 참조는 답변에서 제거	구현
		A07_B02	LLM 스키마 자동 매핑	컬럼명을 표준 용어로 자동 매핑	-	제외
A08	운영 관제·품질 시각화	A08_B01	모니터링	API·대기열·작업자·스냅샷 저장소 상태와 24시간 실행 통계	권한이 없으면 안내 문구만 표시	구현
		A08_B02	품질 센터	WARN·FAIL 결과와 스키마 변화를 테이블 전체에서 조회	점수는 계산하지 않고 검사 결과만 표시	구현
A09	버전 관리	A09_B01	스냅샷·명세 이력	테이블 스냅샷 revision, 명세 개정 이력과 되돌리기	충돌 시 최신 revision 기준으로 거부	구현

- A07_B02는 구현 범위에서 제외하고, 컬럼 대응은 명세의 rename·coalesce로 선언함
- 개인정보 검사는 규칙 기반으로, 버전 관리는 웨어하우스 스냅샷과 명세 개정 이력으로 구현함

서비스 구성도 - 서비스 시나리오



서비스 구성도 - 서비스 시나리오

1. 구성 요소

구성 요소	역할
KPubData Studio	웹 작업공간. 화면과 사용자 흐름 담당. 실행 엔진이 아니며 Builder API만 호출
KPubData Builder	실행·웨어하우스 계층. 명세 검증, Bronze·Silver·Gold 실행, 스냅샷·Manifest 기록, 읽기 전용 SQL 제공
kpubdata	공공 API 접근 라이브러리. 제공 기관별 호출과 표준 레코드 변환 담당
저장소	실행 작업공간(로컬 파일), 웨어하우스 스냅샷(Parquet), SQLite 인덱스
외부 대상	Hugging Face·Kaggle 등 게시 대상, OIDC 인증 제공자(Keycloak)

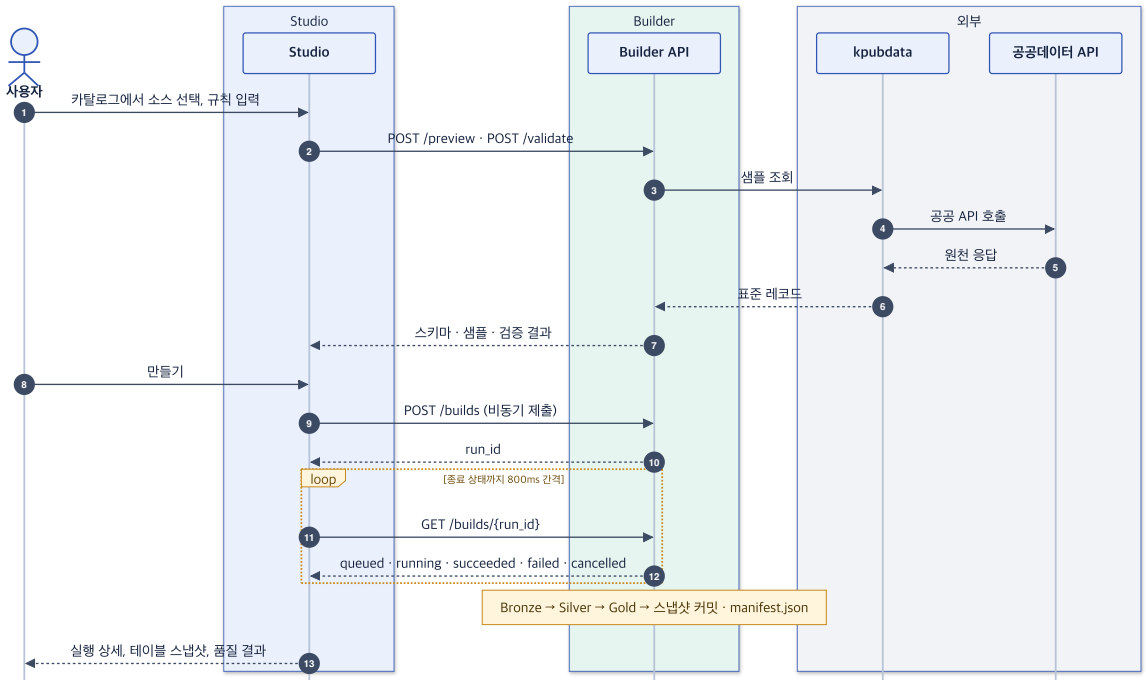
2. 테이블 만들기 (Studio 기반)

- ① 사용자가 Studio 카탈로그에서 소스를 고르고 규칙(BuildSpec)을 구성
- ② Studio가 Builder API에 미리보기와 검증을 요청해 스키마와 샘플 확인
- ③ 만들기를 실행하면 Builder가 Bronze→Silver→Gold로 가공하고 스냅샷과 Manifest 생성
- ④ 사용자는 테이블 상세, SQL 작업 공간, 리포트에서 결과 활용

3. 정기 수집·게시 (일정 기반)

- ① GitHub Actions 일정이 분야별 수집 작업을 주기적으로 시작 (기상 3시간, 관광·의약품 주 1회, 부동산 월초)
- ② kpubdata가 공공 API에서 데이터를 수집하고 게시 스크립트가 정제·패키징
- ③ 검증을 통과한 데이터셋을 Hugging Face에 게시
- ④ 실패하면 이슈를 자동 등록해 담당자가 확인
- 정기 게시는 기존 게시 스크립트 경로로 동작하며, BuildSpec 기반 경로로 순차 이전 중임

서비스 흐름도



- ① 규칙 작성 — 사용자가 카탈로그에서 소스를 선택하고 파라미터와 규칙 입력
- ② 미리보기·검증 — Studio가 `POST /preview`, `POST /validate`를 동시에 요청
- ③ 원천 조회 — Builder가 kpubdata를 통해 공공 API에서 샘플을 받아 표준 레코드로 변환
- ④ 실행 제출 — `POST /builds`로 비동기 제출하고 `run_id` 수신
- ⑤ 진행 확인 — `GET /builds/{run_id}`를 800ms 간격으로 조회 (탭이 숨겨지면 중단)
- ⑥ 결과 확인 — 실행 상세에서 단계별 상태·품질·이벤트 확인, 테이블 스냅샷 활용

인터페이스 정의서 - Builder HTTP API (1/2)

영역	메서드·경로	용도
상태	`GET /healthz` · `/readyz`	생존·준비 상태 확인 (인증 없음)
	`GET /version`	API 계약 버전 조회
	`GET /metrics`	Prometheus 형식 지표 (관리자 전용)
	`GET /monitoring/summary` · `/monitoring/builds`	API·대기열·작업자·저장소 상태, 시간대별 실행 통계
카탈로그·연결	`GET /catalog`	제공 기관과 소스 데이터셋 목록
	`GET /providers`	제공 기관과 설정 상태
	`POST /providers/{p}/test` · `/probe`	연결 확인, 데이터셋별 키 확인
	`GET /providers/{p}/status`	호출자의 자격 증명으로 본 연결 상태
	`GET·PUT·DELETE /providers/{p}/credential`	저장된 자격 증명 조회·등록·삭제
업로드	`POST /uploads`	파일 소스용 파일 업로드
	`GET /uploads` · `GET·DELETE /uploads/{id}`	업로드 목록, 메타데이터 조회, 삭제
명세	`POST /validate`	BuildSpec 검증
	`POST /preview`	저장 없이 소스별 스키마와 샘플 행 반환
실행	`POST /build`	동기 실행. 요청 안에서 파이프라인 수행
	`POST /builds`	비동기 실행 제출. run_id 반환
	`GET /builds`	실행 이력 목록
	`GET /builds/{id}`	작업 상태 조회
	`POST /builds/{id}/cancel`	협력적 취소
	`GET /builds/{id}/manifest`	manifest.json 본문
	`GET /builds/{id}/spec`	실행에 쓰인 BuildSpec 스냅샷
	`GET /builds/{id}/events`	실행 이벤트 타임라인
	`GET /builds/{id}/stages` · `/stages/{stage}`	Bronze·Silver·Gold 단계 상태와 상세
	`GET /builds/{id}/quality`	실행의 품질 결과

인터페이스 정의서 - Builder HTTP API (2/2)

영역	메서드·경로	용도
산출물	GET /artifacts/{id} · /artifacts/{id}/{path}	실행 작업공간의 파일 목록, 파일 내려받기
테이블	GET /datasets · /datasets/{id}	만든 테이블 목록, 요약
	GET /datasets/{id}/runs · /runs/{run}	테이블의 실행 이력
	GET /datasets/{id}/quality/history	품질 이력
	GET /quality/summary · /quality/issues	품질 요약, 조치가 필요한 항목
웨어하우스	GET /warehouse/tables · /tables/{name}	커밋된 테이블, 읽을 수 있는 스냅샷
	GET /warehouse/tables/{name}/profile	스냅샷의 컬럼 프로파일
	POST /warehouse/query	고정한 스냅샷에 읽기 전용 SQL
	POST /warehouse/rows	스냅샷의 한 페이지 행
	POST /warehouse/aggregate	검증된 집계
	POST·GET /warehouse/exports, /exports/{id}/download	질의 결과 내보내기 생성·목록·내려받기
질의·분석	POST /query	실행의 Silver·Gold 표에 읽기 전용 SQL
	GET·POST /analyses	저장된 분석 목록, 스냅샷에 고정해 저장
	POST /analyses/{id}/run · DELETE /analyses/{id}	저장한 스냅샷으로 다시 실행, 삭제
개정 이력	PUT·GET /revisions/{kind}/{doc}	명세·주석의 새 개정 저장, 조회
	GET …/history · POST …/revert	개정 이력, 되돌리기
게시	GET /builds/{id}/publish/readiness	게시 준비 상태와 차단 사유
	POST /builds/{id}/publish	게시 실행
	GET·DELETE /builds/{id}/publish/receipt	게시 영수증 조회·초기화
	POST …/publish/reconcile · GET …/publish/audit	대상과 대조, 감사 기록
관리	GET /admin/config	현재 적용 중인 정책
	GET /admin/runs	전체 사용자의 실행 (메타데이터)
	GET /admin/users, POST …/approve · reject	가입 신청 목록, 승인·거절

인터페이스 정의서 - 공통 규칙

- 계약 버전 1.123.0, 연산 77개. 계약 원본은 `contract/builder-api.yaml` (OpenAPI)
- 인증: `X-API-Key` 또는 `Authorization: Bearer` (OIDC JWT). `/healthz`, `/readyz`만 인증 없이 응답
- 제공 기관 키 전달: `X-Provider-Key` (미리보기·실행·연결 확인), 게시 자격 증명: `X-Publish-Credential`
- Studio는 계약 버전이 1.96.0 이상이고 주 버전이 같을 때만 호환으로 판단
- Studio는 모든 응답을 Zod 스키마로 검증하고, 형식이 다르면 계약 불일치 오류로 처리

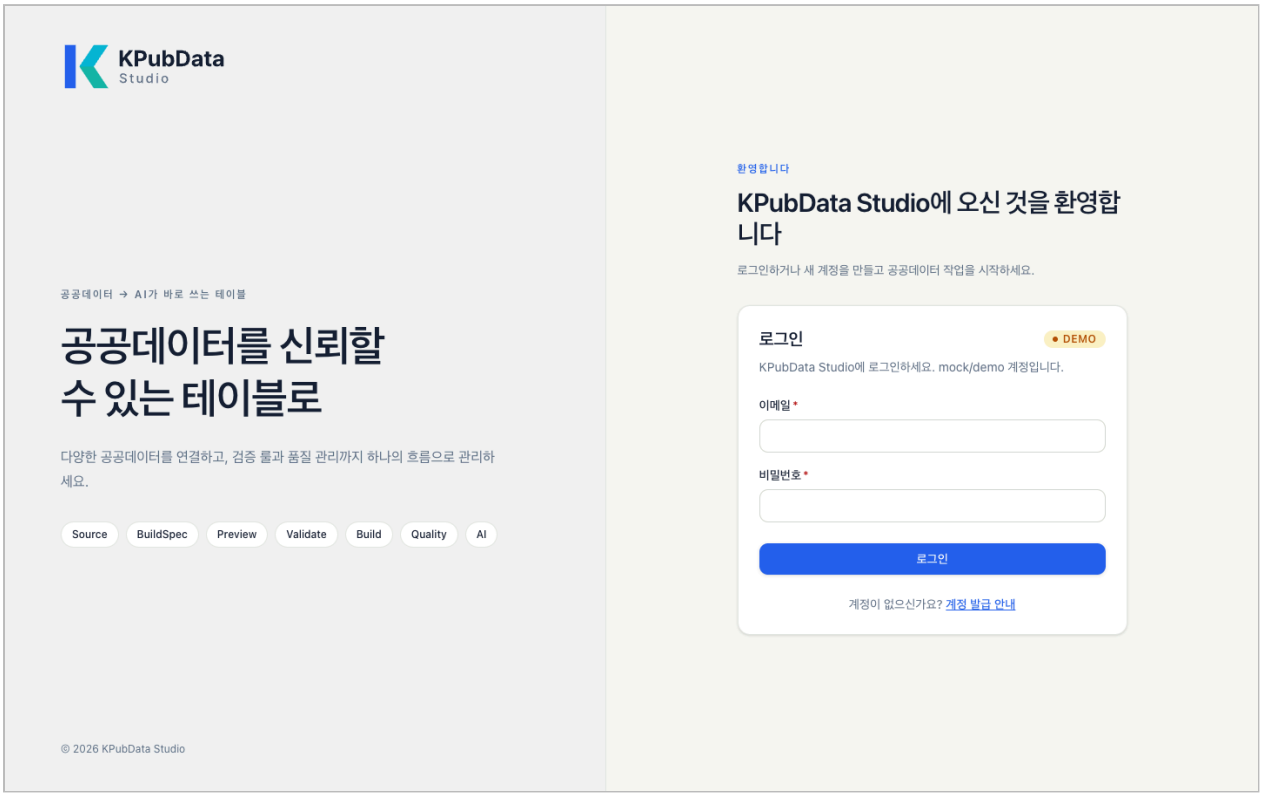
메뉴 구성도



그룹	메뉴	경로	설명
-	홈	`/`	조치가 필요한 테이블과 최근 변경
데이터	카탈로그	`/discover`	공공 API 소스 데이터셋 탐색
	테이블	`/tables`	소스로 만든 테이블 목록과 상세
분석	SQL 작업 공간	`/sql`	테이블 스냅샷에 SQL 실행
	저장된 분석	`/analyses`	스냅샷에 고정해 저장한 분석
	작업대	`/workspace`	최근 작업과 저장한 BuildSpec
	리포트	`/reports`	근거 기반 리포트 작성
운영	갱신 이력	`/refresh-jobs`	테이블을 만들고 갱신한 실행 기록
	품질	`/quality`	품질 검사 결과와 스키마 변화
	모니터링	`/monitoring`	Builder 상태와 실행 통계
	관리	`/admin`	운영 정책·전체 실행·가입 승인 (관리자 전용)
하단	연결	`/connections`	제공 기관 자격 증명
	설정	`/settings`	Builder 연결, 계정, AI 키

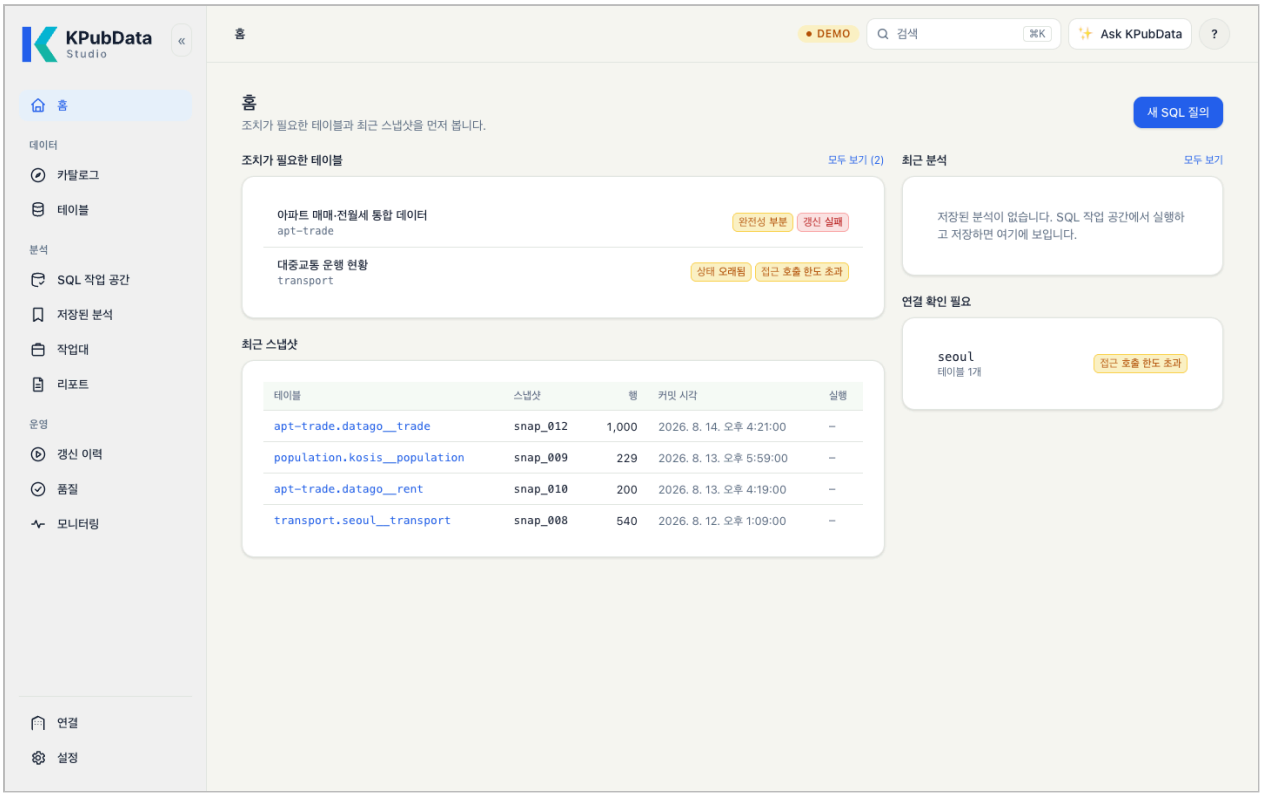
- 테이블 만들기(`/add`)는 메뉴가 아니라 카탈로그·테이블 화면의 동작으로 진입
- 상단 바: 경로 표시, Demo·Beta 배지, 통합 검색(⌘K), Ask KPubData 버튼, 계정 메뉴
- 이전 경로(`/datasets`, `/builds`, `/provider`, `/validate`, `/preview`, `/artifacts`)는 새 경로로 자동 이동

화면 설계서 — 로그인 (1/20)



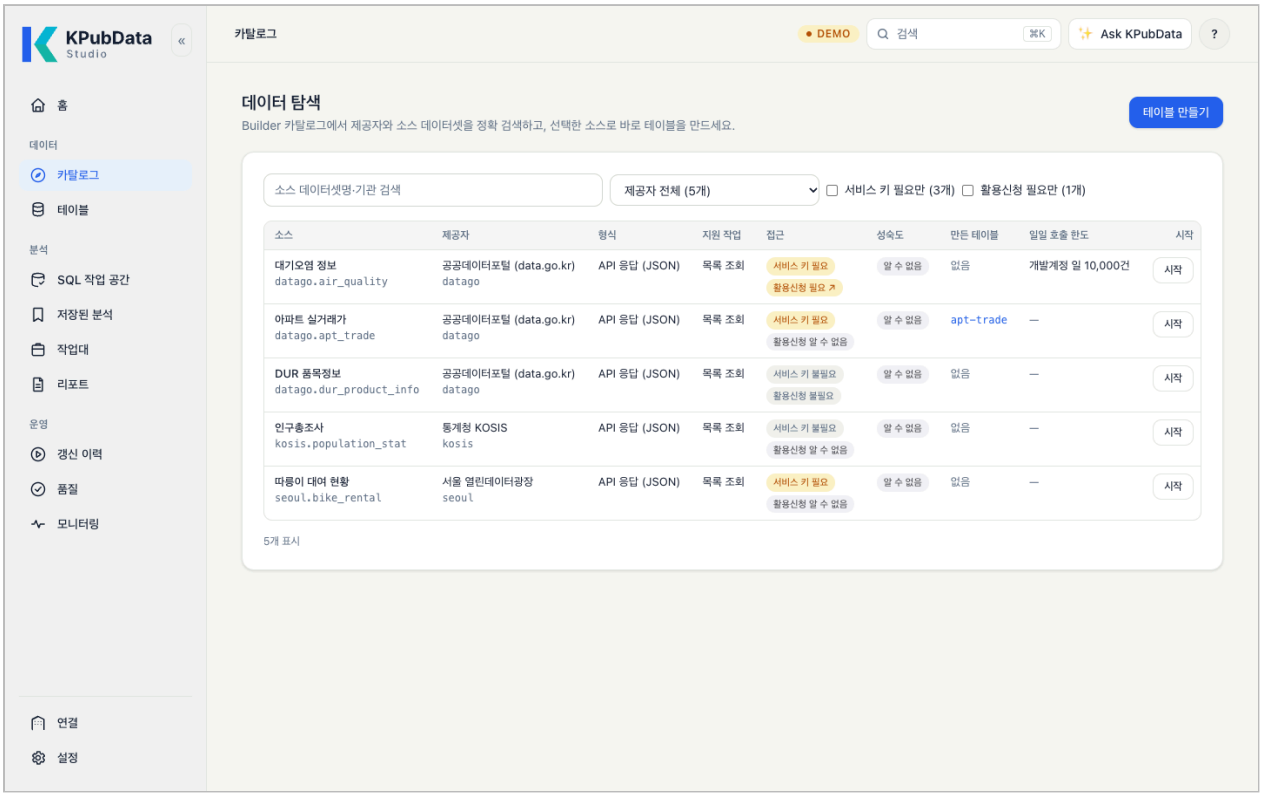
화면 개요	Keycloak 로그인으로 들어가는 진입 화면
주요 UI 구성요소	• 좌측 소개 영역: 제품 문구와 핵심 단계 배지 7종 (Source·BuildSpec·Preview·Validate·Build·Quality·AI) • 우측 로그인 카드: Google 로그인, 이메일 로그인 버튼
상태·상호작용	• 로그인 버튼 클릭 시 Keycloak 인증 화면으로 이동 (OIDC Authorization Code + PKCE) • 데모 모드에서만 이메일·비밀번호 입력 폼 표시 • 인증된 사용자는 원래 가려던 화면으로 이동
관련 API	`Keycloak 인증 서버 (Builder API 호출 없음)`

화면 설계서 — 홈 (2/20)



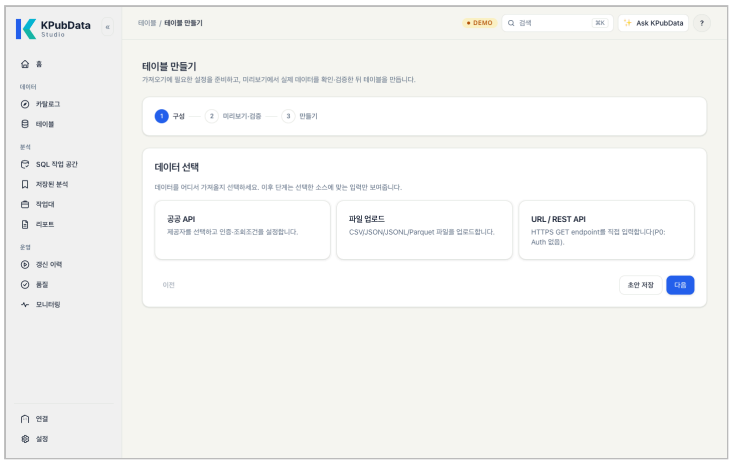
화면 개요	조치가 필요한 테이블과 최근 변경을 먼저 보여주는 시작 화면
주요 UI 구성요소	• 조치가 필요한 테이블 (최대 6개) • 최근 스냅샷과 이를 만든 실행 (최대 5개) • 최근 저장한 분석, 연결 문제 • 상단 '새 SQL 쿼리' 버튼
상태·상호작용	• 테이블과 실행이 하나도 없으면 첫 실행 체크리스트(이용 신청 → 키 등록 → 키 확인 → 첫 테이블) 표시 • 웨어하우스가 없는 배포에서는 최근 스냅샷 대신 최근 실행 표시 • 모든 항목은 한 번의 클릭으로 해당 테이블·분석으로 이동
관련 API	`GET /datasets, GET /builds, GET /warehouse/tables, GET /analyses`

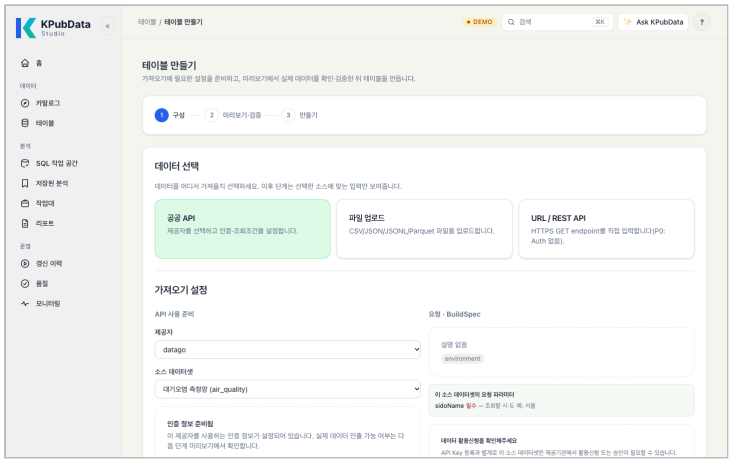
화면 설계서 — 카탈로그 (3/20)



화면 개요	Builder가 제공하는 소스 데이터셋을 비교 표로 찾는 화면
주요 UI 구성요소	• 검색창, 제공 기관 선택, '서비스 키 필요'·'활용 신청 필요' 체크박스 • 비교 표: 소스, 제공 기관, 표현, 지원 동작, 접근 조건, 만든 테이블, 호출 한도, 시작 • 상단 '테이블 만들기' 버튼
상태·상호작용	• 필터 조건은 주소(`?q=&provider=`)에 남아 공유 가능 • 행의 시작 버튼 클릭 시 소스가 채워진 채 테이블 만들기로 이동 • 로딩·오류(다시 시도)·결과 없음 상태를 구분해 표시
관련 API	`GET /catalog, GET /datasets`

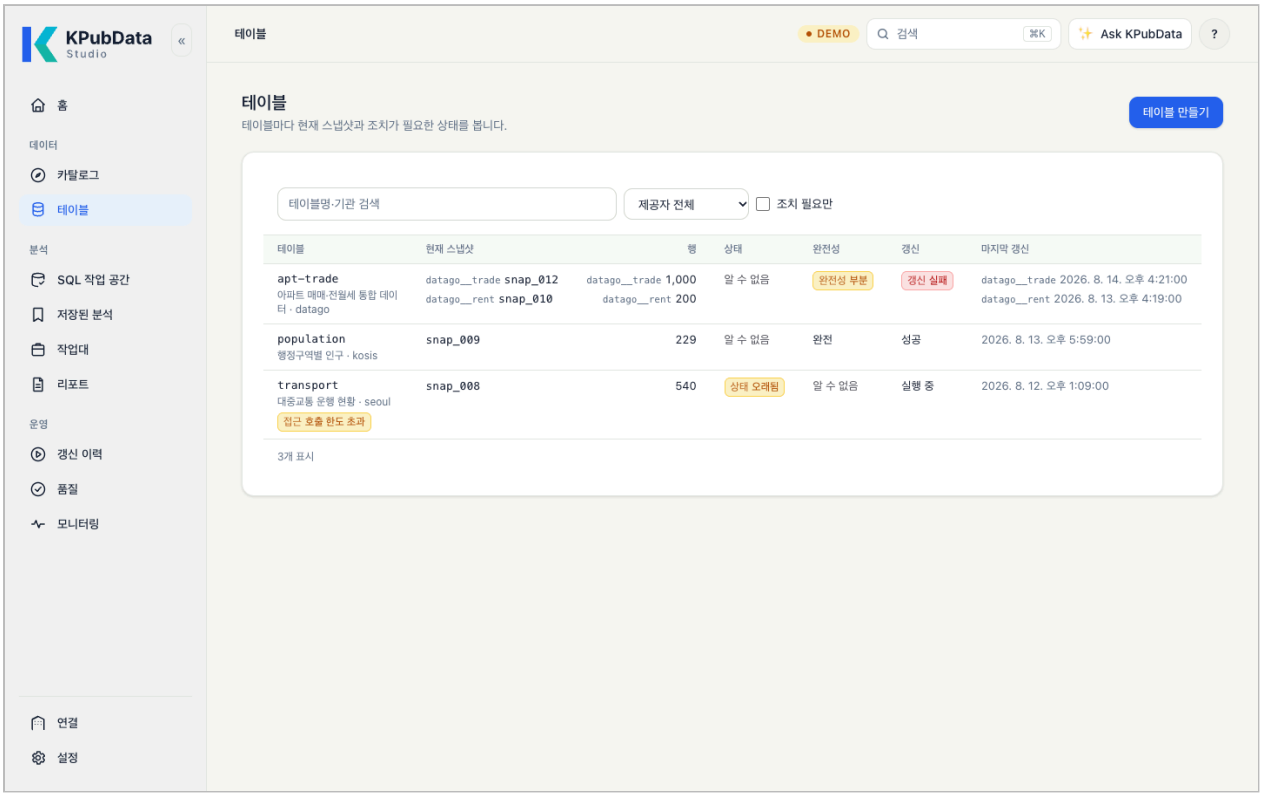
화면 설계서 — 테이블 만들기 (4/20)





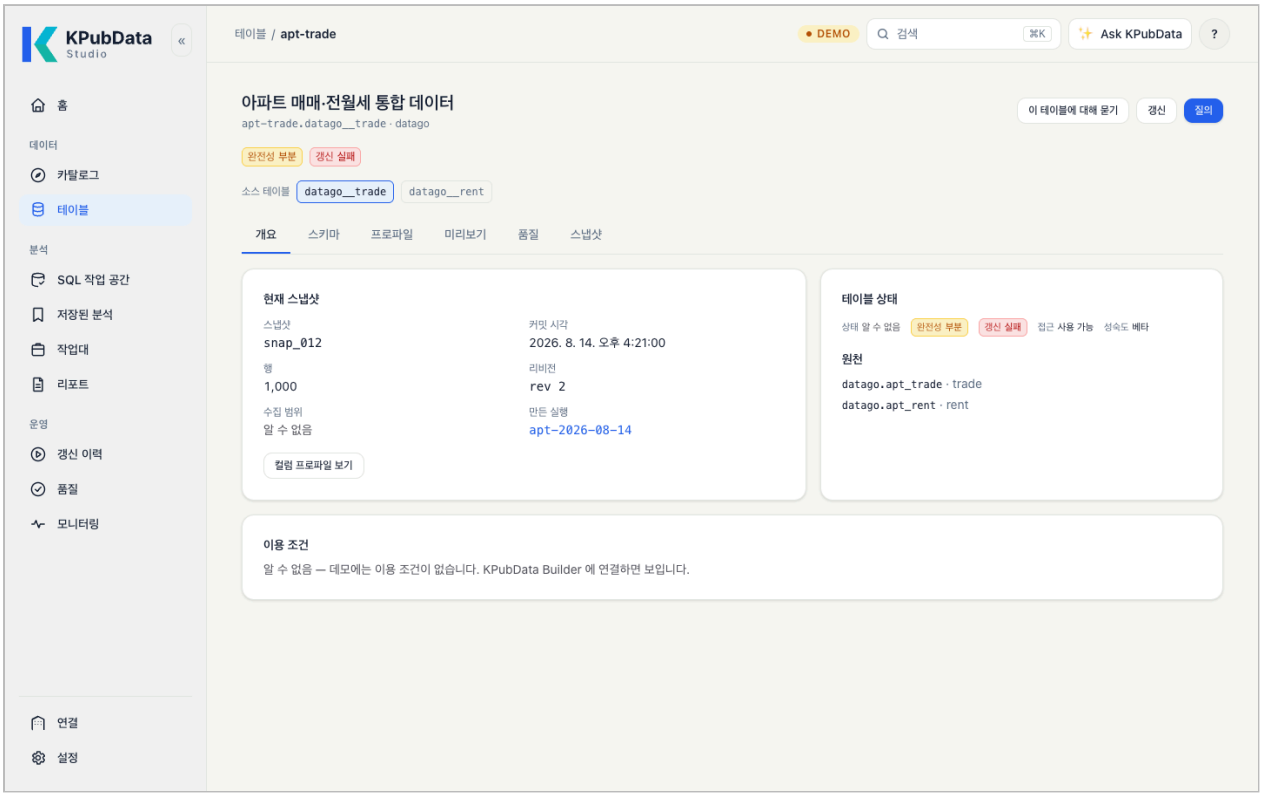
화면 개요	소스 선택부터 생성까지 3단계로 진행하는 마법사
주요 UI 구성요소	• 단계 표시: 구성 → 미리보기·검증 → 만들기 • 1단계: 소스 종류 카드 3종(공공 API, 파일 업로드, URL / REST API)과 가져오기 설정 • 2단계: 소스별 스키마·샘플과 검증 결과 • 3단계: 최종 명세, 테이블 이름, 같은 이름의 테이블 안내 • 하단: 이전, 초안 저장, 다음
상태·상호작용	• 소스 종류를 고르지 않았거나 명세 오류가 있으면 다음 단계 차단 • 자격 증명이 없으면 연결 화면으로 안내 후 복귀 • 미리보기가 오래되면 만들기 차단 • 제출이 수락되면 초안을 정리하고 실행 상세로 이동
관련 API	`POST /preview, POST /validate, POST /uploads, GET /datasets/{id}, POST /builds`

화면 설계서 — 테이블 (5/20)



화면 개요	만든 테이블을 현재 스냅샷과 상태로 확인하는 목록 화면
주요 UI 구성요소	• 검색창, 제공 기관 선택, '조치 필요만' 필터 • 표: 테이블, 스냅샷, 행 수, 상태, 완전성, 갱신, 마지막 갱신
상태·상호작용	• 행 클릭 시 테이블 상세로 이동 • 웨어하우스가 없으면 스냅샷 열을 숨기고 안내 한 줄 표시 • 테이블이 없으면 테이블 만들기로 가는 안내 표시
관련 API	`GET /datasets, GET /warehouse/tables`

화면 설계서 — 테이블 상세 (6/20)



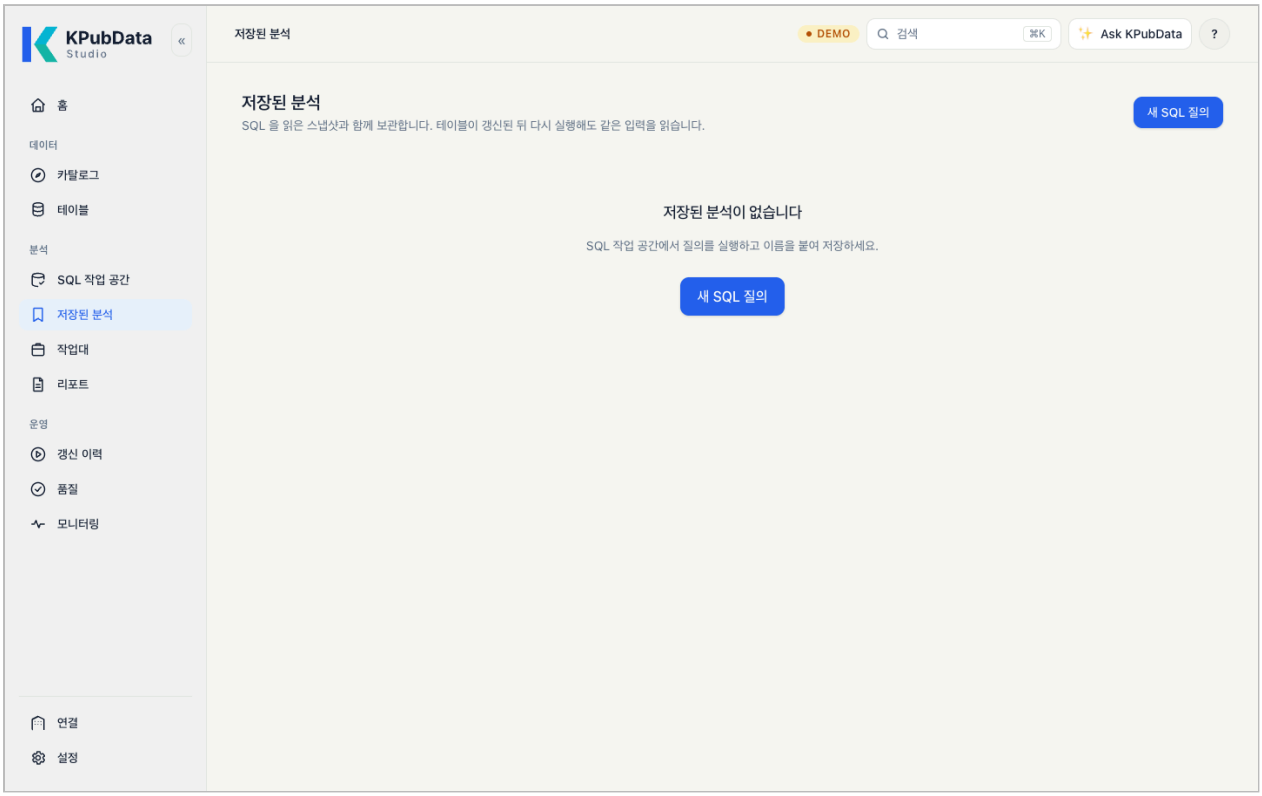
화면 개요	테이블 하나의 스냅샷·스키마·품질을 탭으로 확인하는 화면
주요 UI 구성요소	• 탭: 개요, 스키마, 프로파일, 미리보기, 품질, 스냅샷 • 상단 동작: 이 테이블에 대해 질문, 갱신, 쿼리 • 소스가 여러 개인 테이블은 소스 전환 제공
상태·상호작용	• 과거 스냅샷을 볼 때 '현재로 돌아가기' 버튼 표시 • 스냅샷 탭에서 실행 상세·명세 편집·게시로 이동 • 커밋된 스냅샷이 없으면 실행 기준 보기(개요·스키마·미리보기·품질·실행)로 표시
관련 API	`GET /warehouse/tables/{name}, GET /warehouse/tables/{name}/profile, POST /warehouse/rows, GET /datasets/{id}/quality/history`

화면 설계서 — SQL 작업 공간 (7/20)



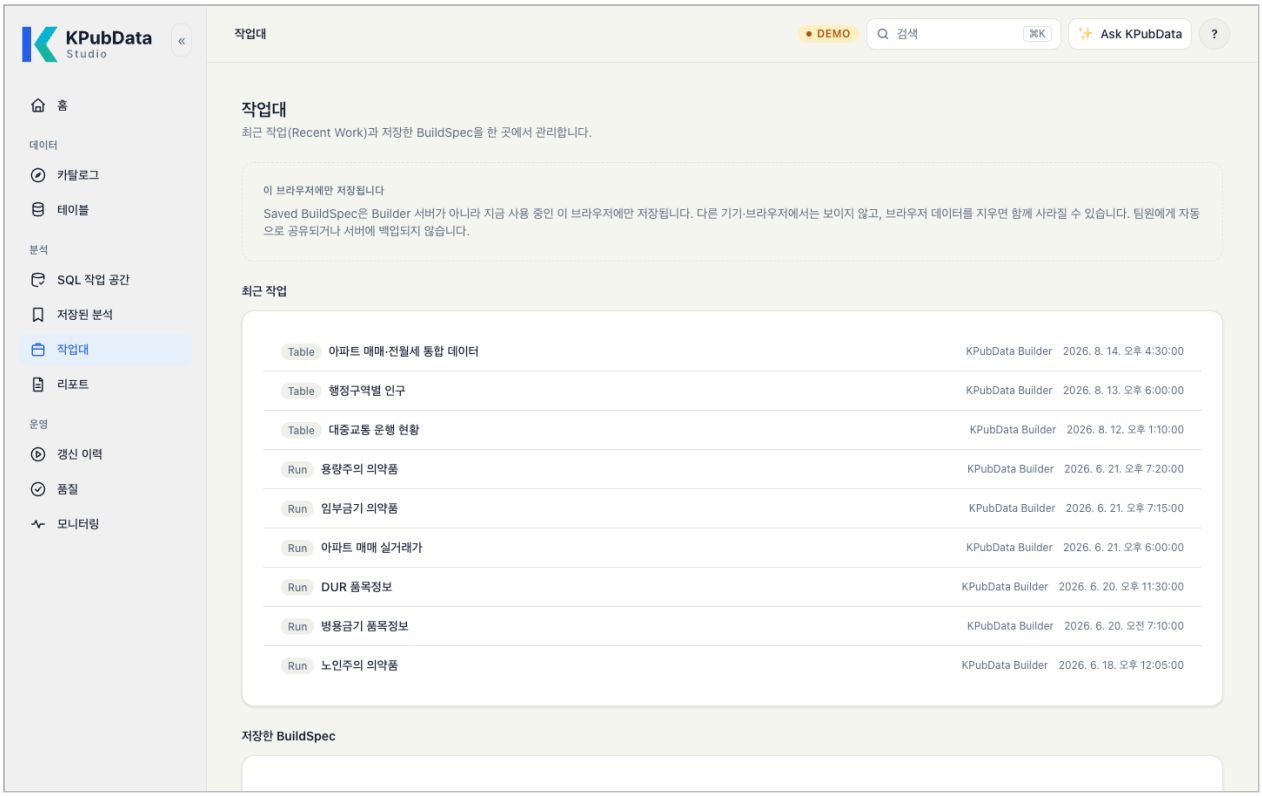
화면 개요	테이블 스냅샷 하나에 읽기 전용 SQL을 실행하는 화면
주요 UI 구성요소	• 테이블 탐색기 (테이블 → 소스 → 컬럼) • 스냅샷 선택(기본값 현재), SQL 입력창, 실행 버튼 • 결과 표(`table@snapshot · rev N`), 분석 저장, 집계 차트, 결과 내보내기
상태·상호작용	• ⌘/Ctrl+Enter로 실행 • 쿼리 하나에 테이블 하나만 조회 • 기본 문장은 `SELECT * FROM dataset LIMIT 100` • SQL 안전성 검사는 Builder가 수행
관련 API	`POST /warehouse/query`, `POST /warehouse/aggregate`, `POST /warehouse/exports`, `POST /analyses`, `POST /query`

화면 설계서 — 저장된 분석 (8/20)



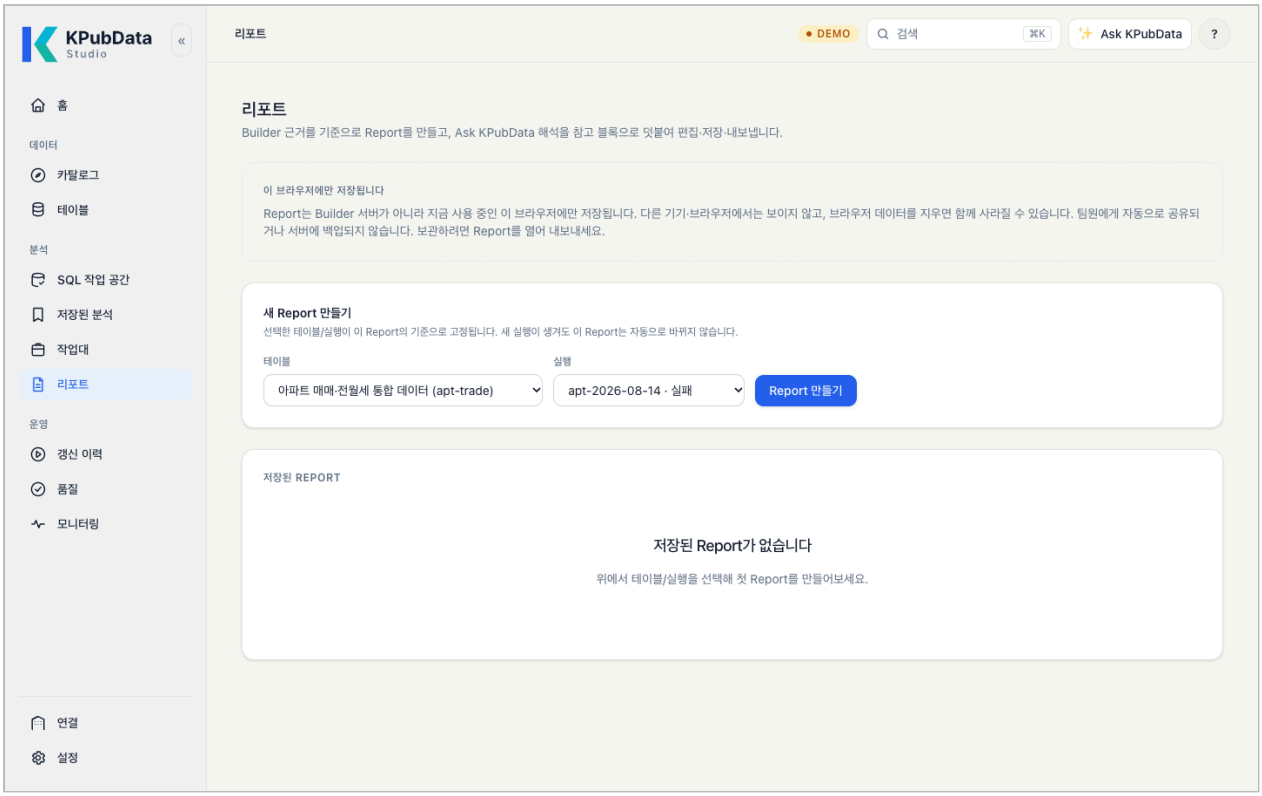
화면 개요	스냅샷에 고정해 저장한 분석을 다시 실행하는 화면
주요 UI 구성요소	• 분석 카드: 이름, `table@snapshot`, 저장 정보 • 버튼: 다시 실행, SQL 작업 공간에서 열기, 삭제
상태·상호작용	• 저장된 분석이 없으면 SQL 작업 공간으로 가는 안내 표시 • 삭제는 확인 후 진행
관련 API	`GET /analyses, POST /analyses/{id}/run, DELETE /analyses/{id}`

화면 설계서 — 작업대 (9/20)



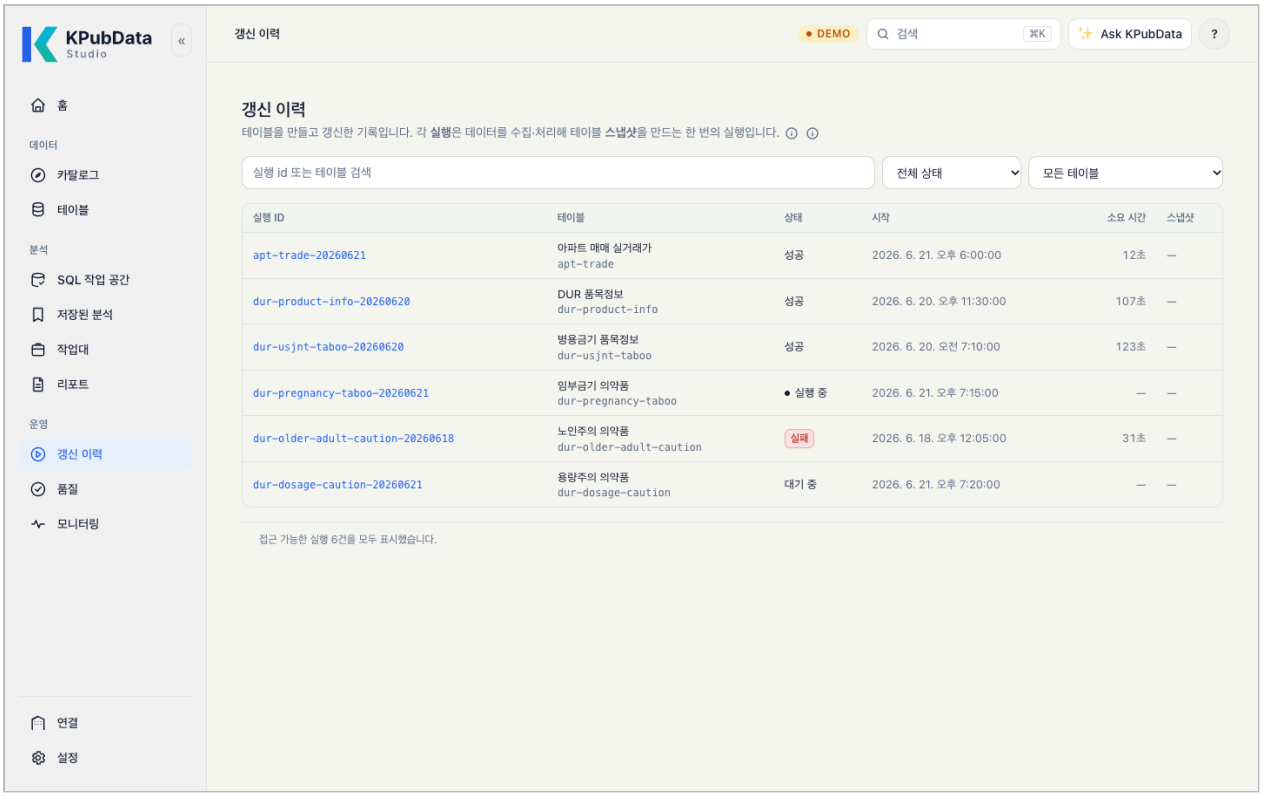
화면 개요	최근 작업과 저장한 BuildSpec을 한곳에서 관리하는 화면
주요 UI 구성요소	• 최근 작업 목록 (테이블·실행·리포트·저장한 명세) • 저장한 BuildSpec: 이름 변경, 복제, 삭제
상태·상호작용	• 저장한 BuildSpec은 이 브라우저에만 보관됨을 배지로 표시 • 저장한 명세를 열면 테이블 만들기로 이동
관련 API	`GET /datasets, GET /builds` (저장한 BuildSpec은 브라우저 저장소)

화면 설계서 — 리포트 (10/20)



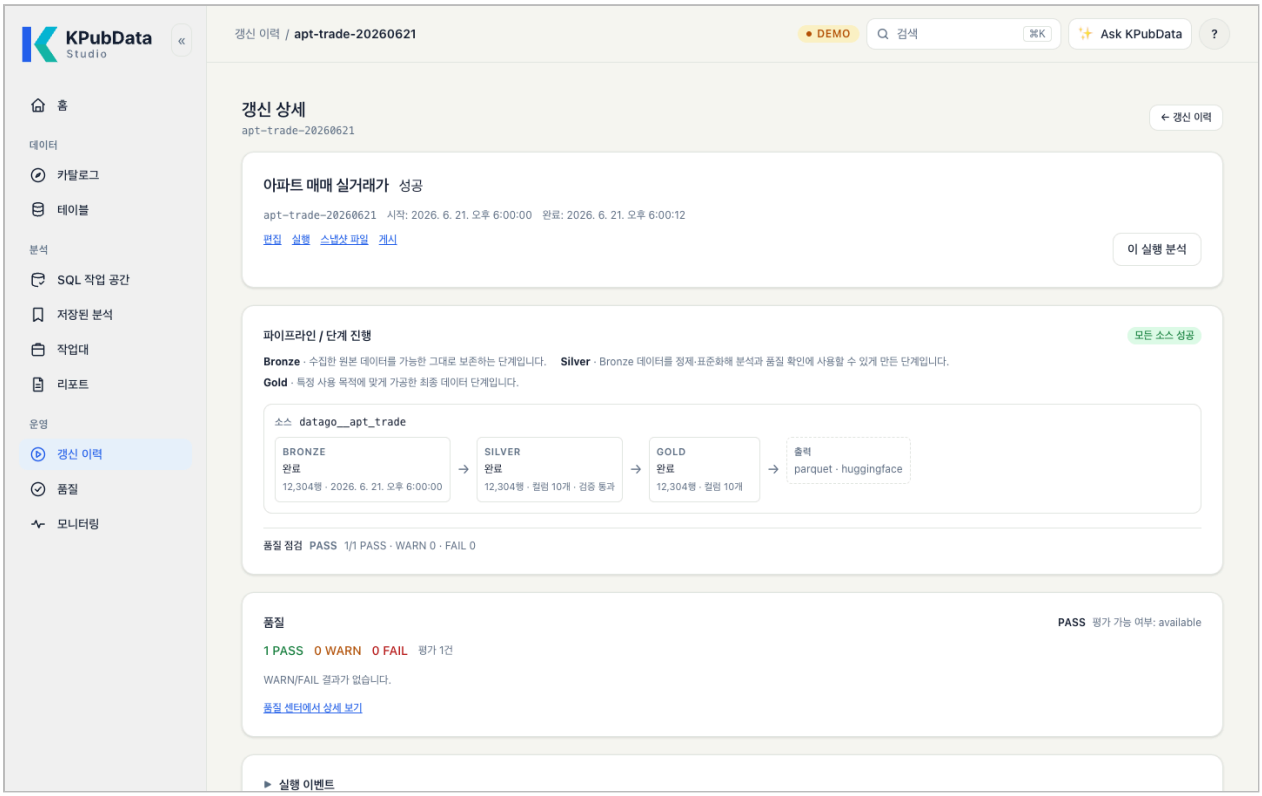
화면 개요	실행 근거를 기준으로 리포트를 만들고 편집하는 화면
주요 UI 구성요소	• 새 리포트 만들기: 테이블·실행 선택 • 리포트 목록: 열기, 이름 변경, 복제, 삭제 • 편집기: 근거 영역(1~6), AI 분석(7), 사용자 메모(8), 자동 저장
상태·상호작용	• 리포트는 이 브라우저에만 저장됨을 배지로 표시 • 근거 새로 고침, Markdown·HTML 내려받기, 인쇄 제공 • 선택한 실행이 기준으로 고정되며 새 실행이 생겨도 자동으로 바뀌지 않음
관련 API	`GET /datasets, GET /datasets/{id}/runs` 등 실행 근거 조회 (리포트 본문은 브라우저 저장소)

화면 설계서 — 갱신 이력 (11/20)



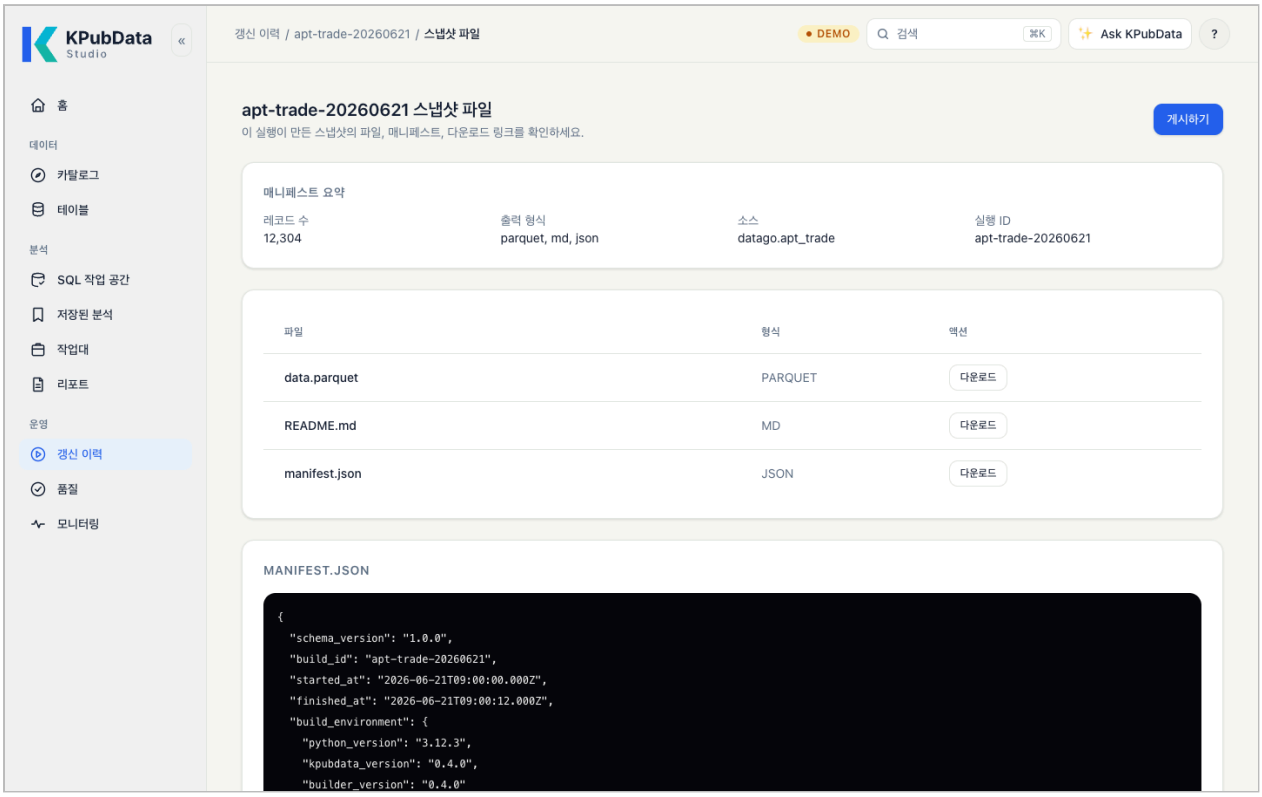
화면 개요	테이블을 만들고 갱신한 실행을 한 표로 조회하는 화면
주요 UI 구성요소	• 검색창, 상태 필터(전체·성공·실패·실행 중·대기·취소), 테이블 필터 • 표: 실행 ID, 테이블, 상태, 시작, 소요 시간, 스냅샷
상태·상호작용	• 행 클릭 시 실행 상세로 이동 • '더 보기'로 이전 실행 추가 조회 • 필터 조건은 주소에 남음
관련 API	`GET /builds?limit=`

화면 설계서 — 실행 상세 (12/20)



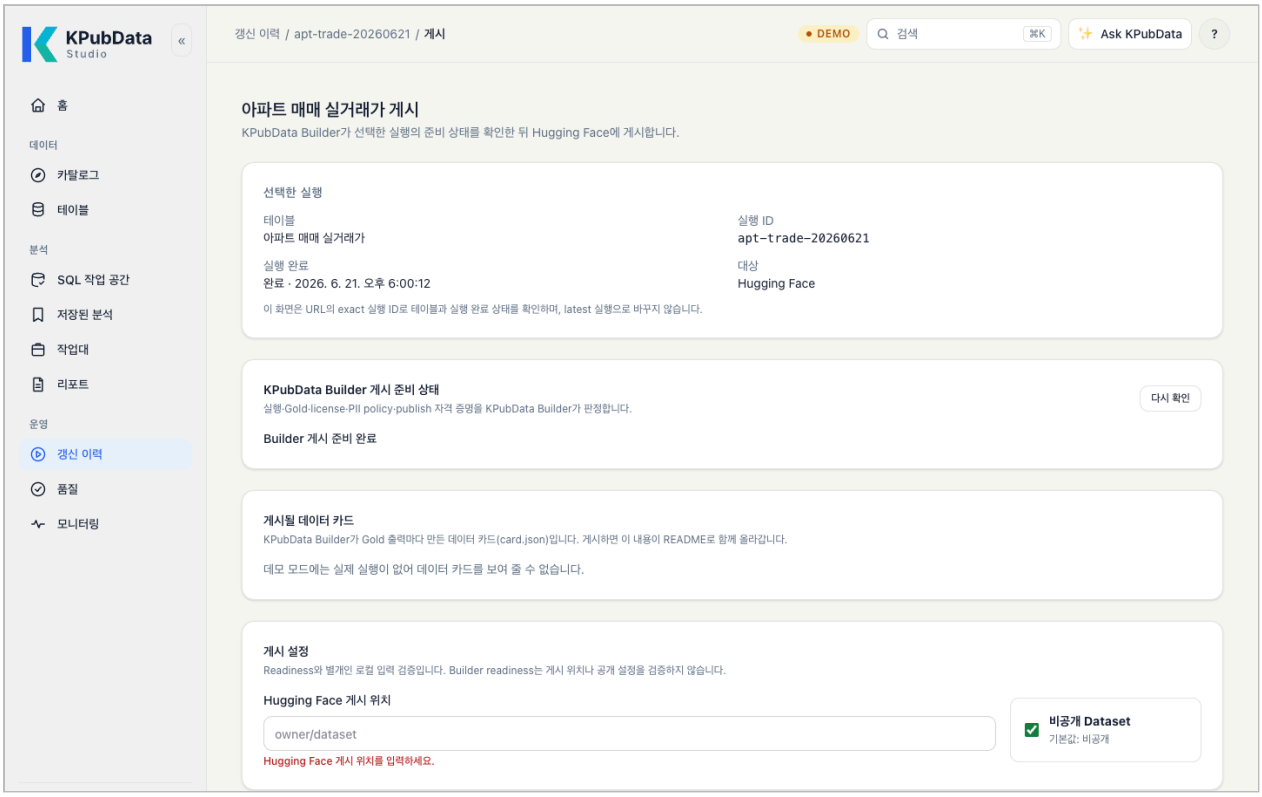
화면 개요	실행 하나의 단계별 결과와 품질, 이벤트를 확인하는 화면
주요 UI 구성요소	• 상단: 제목, 상태, 취소 버튼(진행 중일 때), 편집·산출물·게시 링크 • 소스별 진행: Bronze → Silver → Gold → 출력 • 품질 결과, 실패 근거, 실행 이벤트 타임라인, 명세 스냅샷
상태·상호작용	• 진행 중에는 800ms 간격으로 상태 갱신, 이벤트는 3초 간격 • 취소는 단계 경계에서 반영되며 확인 후 진행 • 실행 없음·조회 실패·권한 없음을 각각 다른 안내로 표시
관련 API	`GET /builds/{id}, GET /builds/{id}/stages, GET /builds/{id}/quality, GET /builds/{id}/events, GET /builds/{id}/spec, POST /builds/{id}/cancel`

화면 설계서 — 산출물 (13/20)



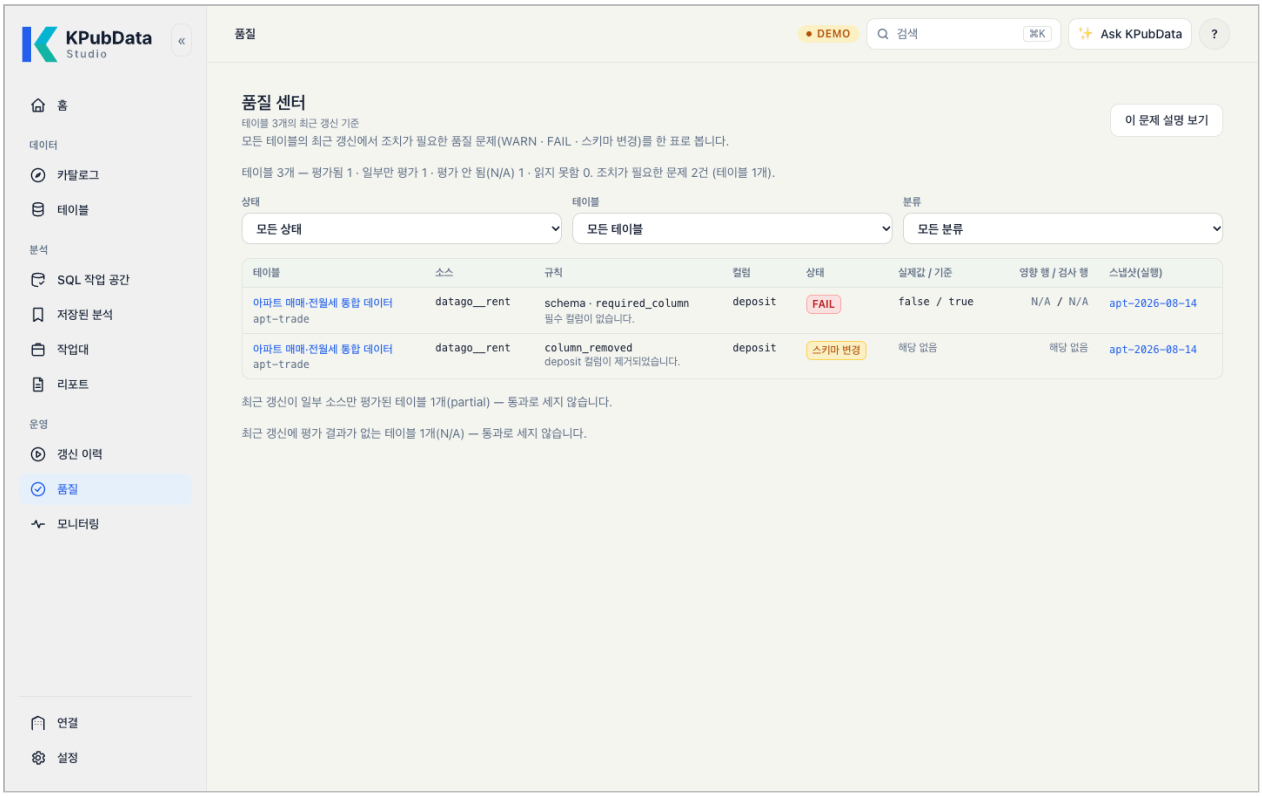
화면 개요	실행이 만든 파일과 Manifest를 확인하는 화면
주요 UI 구성요소	• Manifest 요약, 내려받을 수 있는 파일 목록, manifest.json 원문
상태·상호작용	• 영역별로 따로 불러와 한 영역의 실패가 다른 영역을 막지 않음 • 파일 내려받기는 게시와 구분됨
관련 API	`GET /builds/{id}/manifest, GET /artifacts/{id}, GET /artifacts/{id}/{path}`

화면 설계서 — 게시 (14/20)



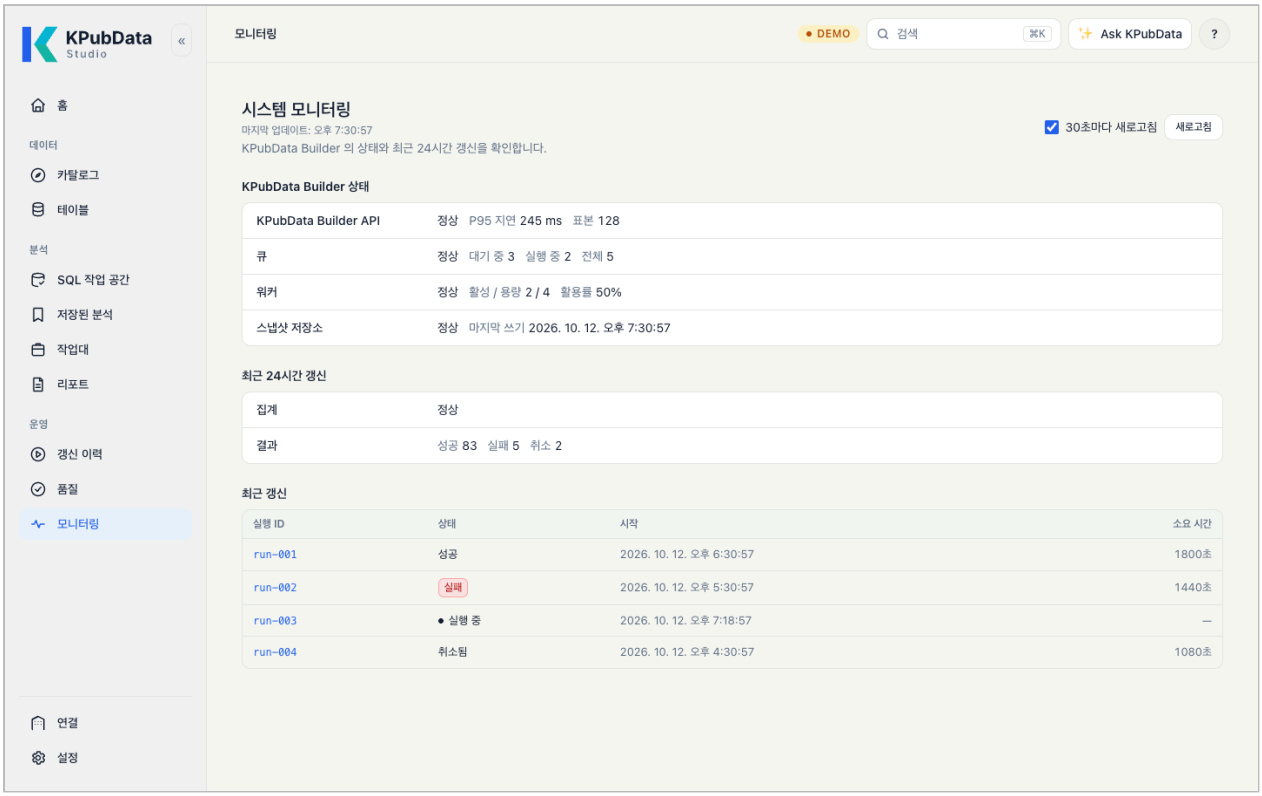
화면 개요	실행 결과를 Hugging Face에 게시하는 화면
주요 UI 구성요소	• 게시 준비 상태 (차단 사유·경고) • 데이터 카드 미리보기 • 대상 저장소(`owner/dataset`), 비공개 여부 • 검토·확인 후 '지금 게시'
상태·상호작용	• 차단 사유가 있으면 게시 버튼 비활성 • 비상업 조건 데이터는 별도 확인 절차 • 결과가 불명확하면 자동 재시도 없이 복구 패널(대조·영수증 초기화) 제공
관련 API	`GET /builds/{id}/publish/readiness, POST /builds/{id}/publish, POST /builds/{id}/publish/reconcile, DELETE /builds/{id}/publish/receipt`

화면 설계서 — 품질 (15/20)



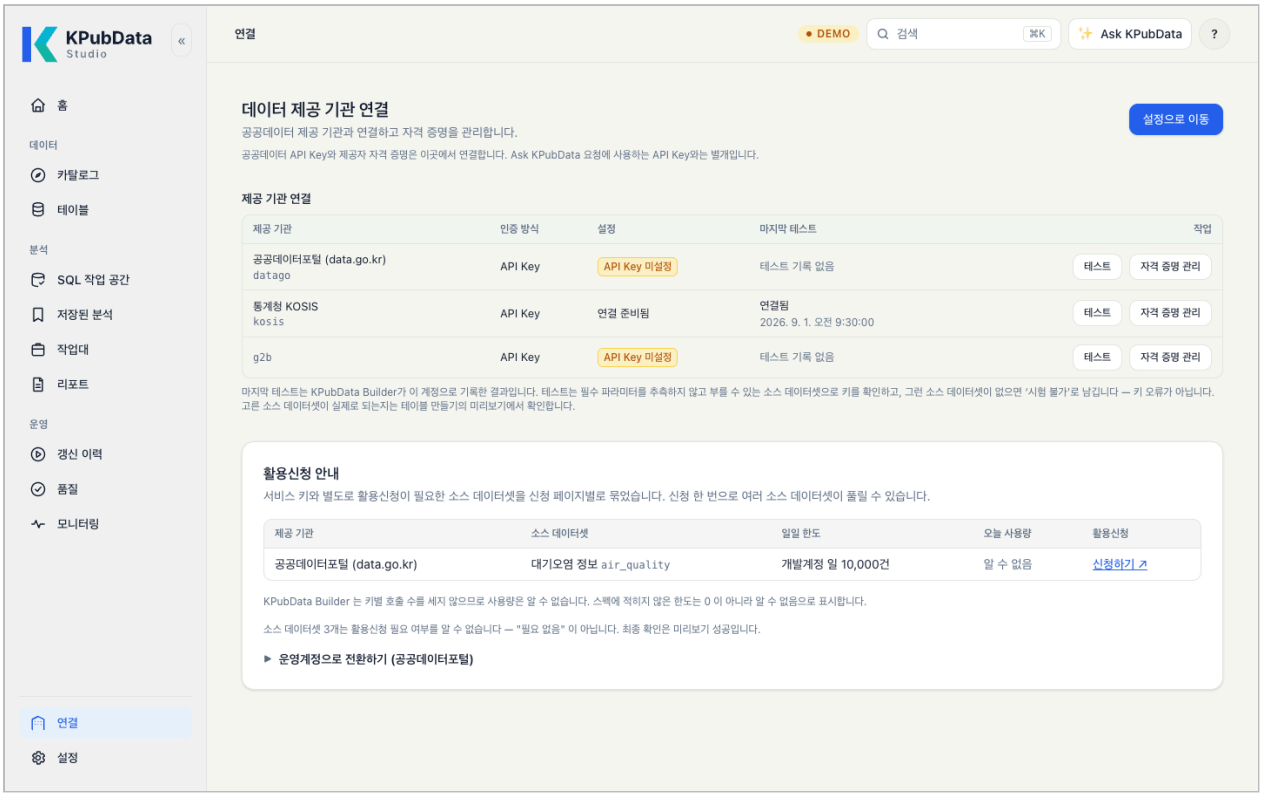
화면 개요	테이블 전체의 품질 경고와 스키마 변화를 모아 보는 화면
주요 UI 구성요소	• 필터: 상태, 테이블, 분류 • 표: 테이블, 소스, 규칙, 컬럼, 상태, 실제값 / 기준값, 영향 행 수, 스냅샷
상태·상호작용	• 점수 대신 검사 결과(PASS·WARN·FAIL·N/A)만 표시 • 각 행은 테이블 상세의 품질 탭으로 연결 • '도우미로 분석' 버튼 제공
관련 API	`GET /quality/summary, GET /quality/issues`

화면 설계서 — 모니터링 (16/20)



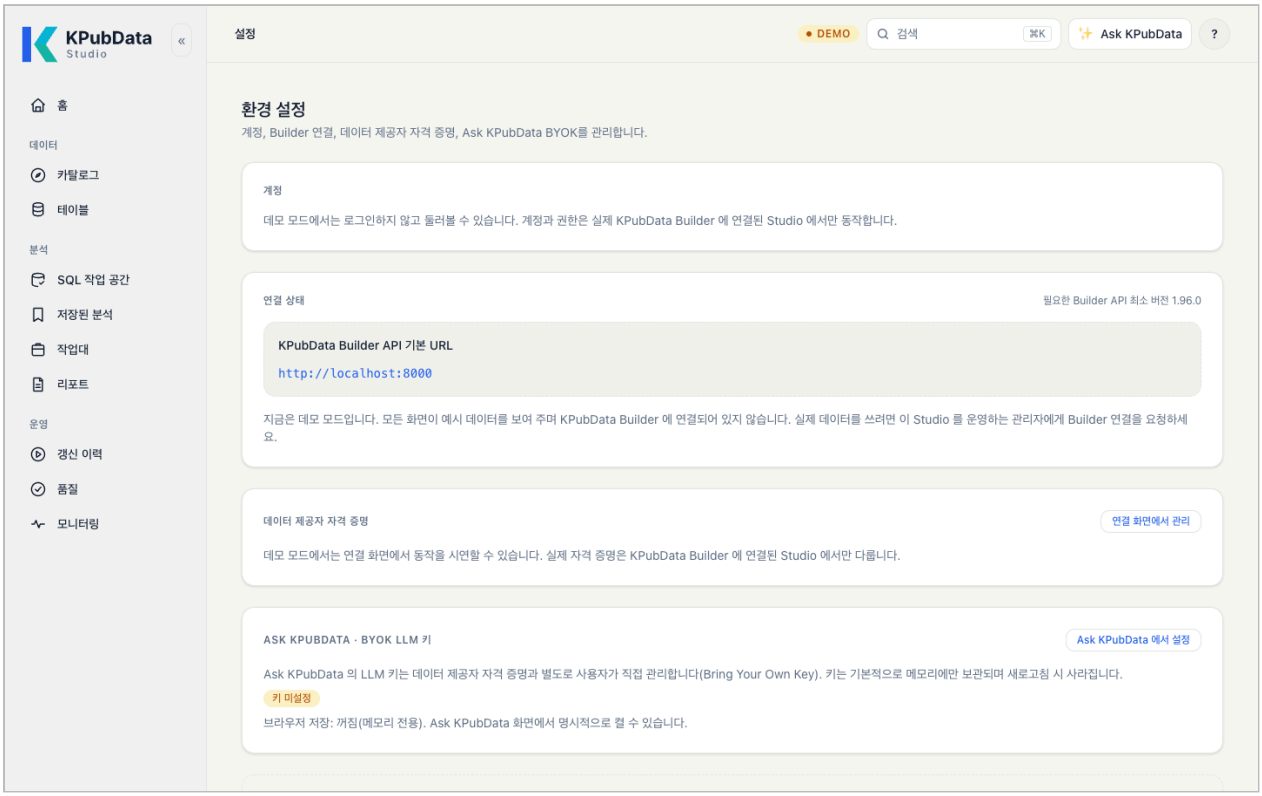
화면 개요	Builder 상태와 최근 24시간 실행 현황을 확인하는 화면
주요 UI 구성요소	• Builder 상태: API, 대기열, 작업자, 스냅샷 저장소 • 최근 24시간 실행 통계, 최근 갱신 목록 • 자동 새로 고침 체크박스, 수동 새로 고침 버튼
상태·상호작용	• 30초 간격으로 조회하며 탭이 보이지 않으면 중단 • 권한이 없으면 안내 한 줄만 표시 • 일부 값만 받지 못하면 저하 상태 배너 표시
관련 API	`GET /monitoring/summary, GET /monitoring/builds`

화면 설계서 — 연결 (17/20)



화면 개요	공공데이터 제공 기관의 자격 증명을 관리하는 화면
주요 UI 구성요소	• 표: 제공 기관, 인증 방식, 설정 여부, 마지막 확인, 동작 • 선택한 기관의 자격 증명 패널, 키 확인, 활용 신청 안내
상태·상호작용	• 단일 사용자 배포: 키를 저장·삭제 • 다중 사용자 배포: 키를 이번 세션에만 사용 (메모리 보관) • 키 원문은 화면에 표시하지 않음
관련 API	`GET /providers, POST /providers/{p}/test, POST /providers/{p}/probe, GET·PUT·DELETE /providers/{p}/credential`

화면 설계서 — 설정 (18/20)



화면 개요	Builder 연결과 계정, AI 키 상태를 확인하는 화면
주요 UI 구성요소	• 연결: Builder 주소와 계약 버전 확인 • 개인정보 안내, 계정, 제공 기관 자격 증명 요약 • Ask KPubData 키(BYOK) 상태
상태·상호작용	• Builder 계약 버전이 최소 요건(1.96.0)보다 낮으면 모든 화면에 경고 배너 • AI 키는 기본적으로 메모리에만 보관
관련 API	`GET /version, GET /providers`

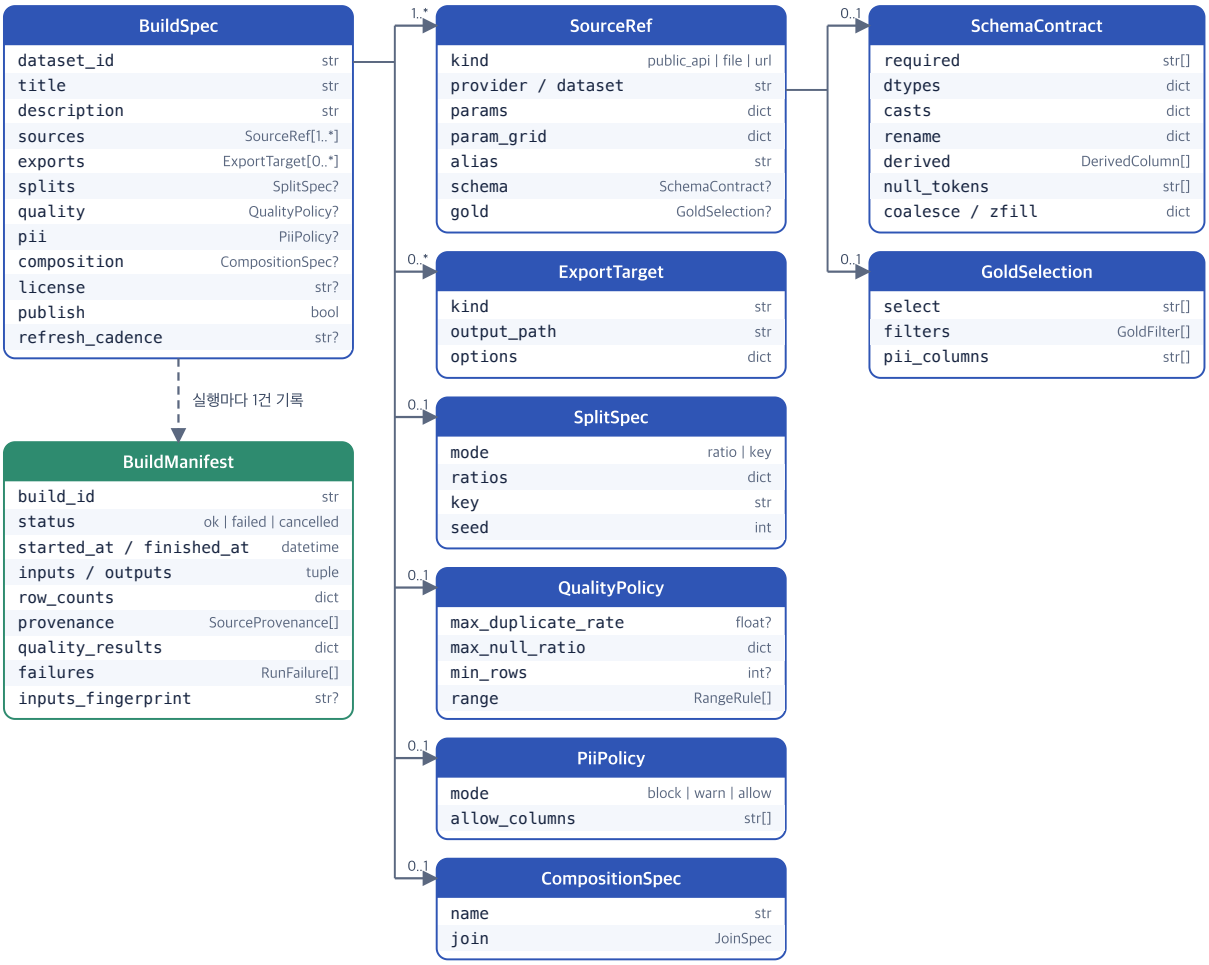
화면 설계서 — Ask KPubData (19/20)

화면 개요	현재 화면의 테이블·실행·품질 문맥으로 질문하고 다음 동작까지 이어 가는 AI 도우미
주요 UI 구성요소	• 문맥 표시(테이블·실행·품질·단계), 추천 질문, 관련 데이터셋 • 답변, 근거 목록, 생성된 SQL(실행·복사), 제안 동작(승인·거절) • 키 입력 폼: API 키, 모델, 주소
상태·상호작용	• 오른쪽 서랍으로 열리며 전체 화면(`/assistant`)도 제공 • SQL은 사용자가 실행을 눌렀을 때만 Builder로 전달 • 문맥이 바뀐 뒤의 제안 동작은 실행할 수 없음을 표시
관련 API	`GET /catalog`, `GET /datasets/*`, `GET /builds/*`, `POST /query`, `POST /validate`, 외부 LLM API

화면 설계서 — 관리 (20/20)

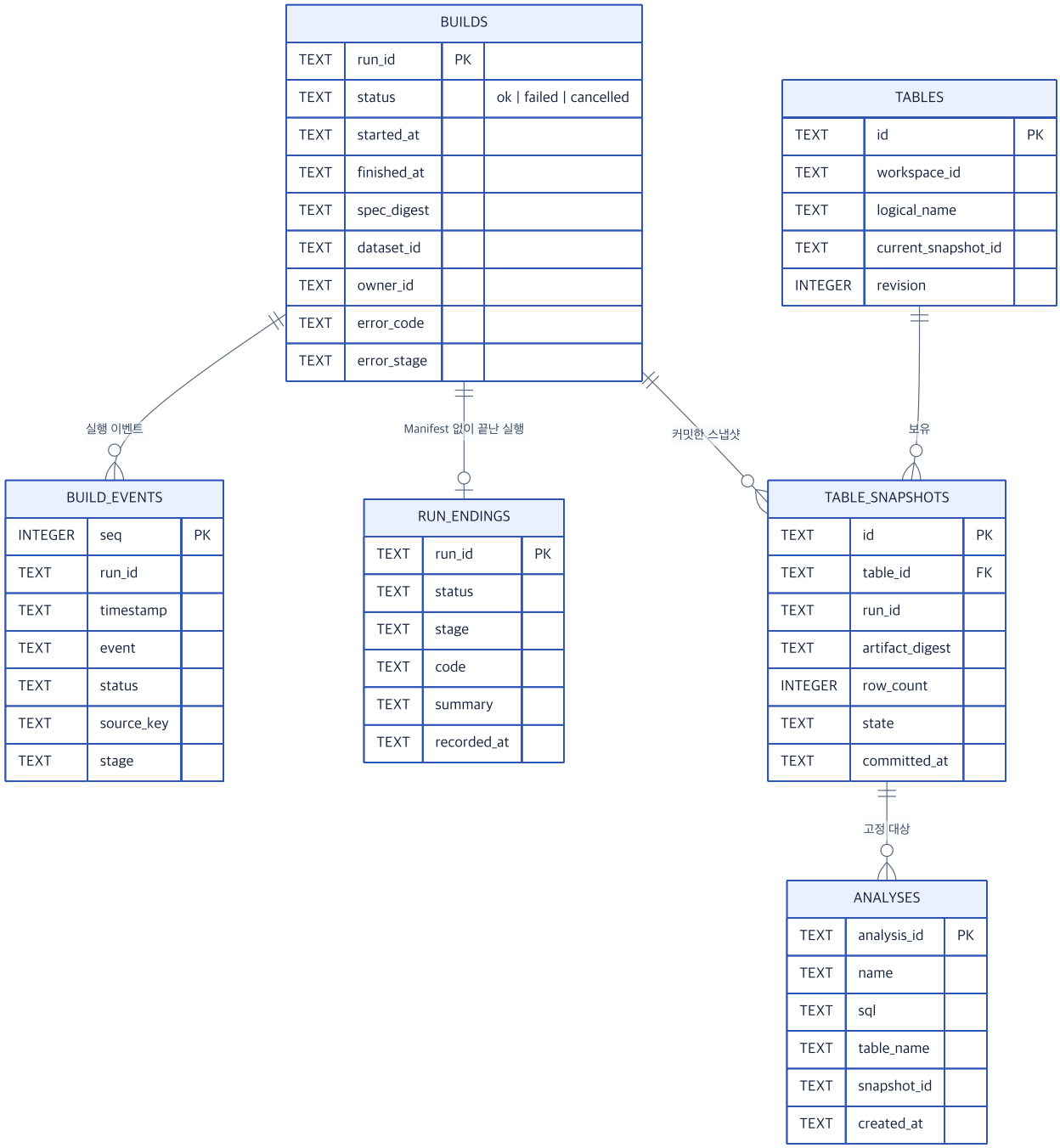
화면 개요	관리자가 운영 정책과 전체 실행, 가입 신청을 확인하는 화면
주요 UI 구성요소	• 현재 적용 중인 정책 • 전체 사용자의 실행 목록 (메타데이터만) • 가입 신청 목록: 승인, 거절
상태·상호작용	• 관리자에게만 메뉴 표시 • 카드별로 로딩·권한 없음·미지원·실패 상태를 따로 표시
관련 API	`GET /admin/config, GET /admin/runs, GET /admin/users, POST /admin/users/{id}/approve · reject`

엔티티 관계도 - 핵심 도메인 모델



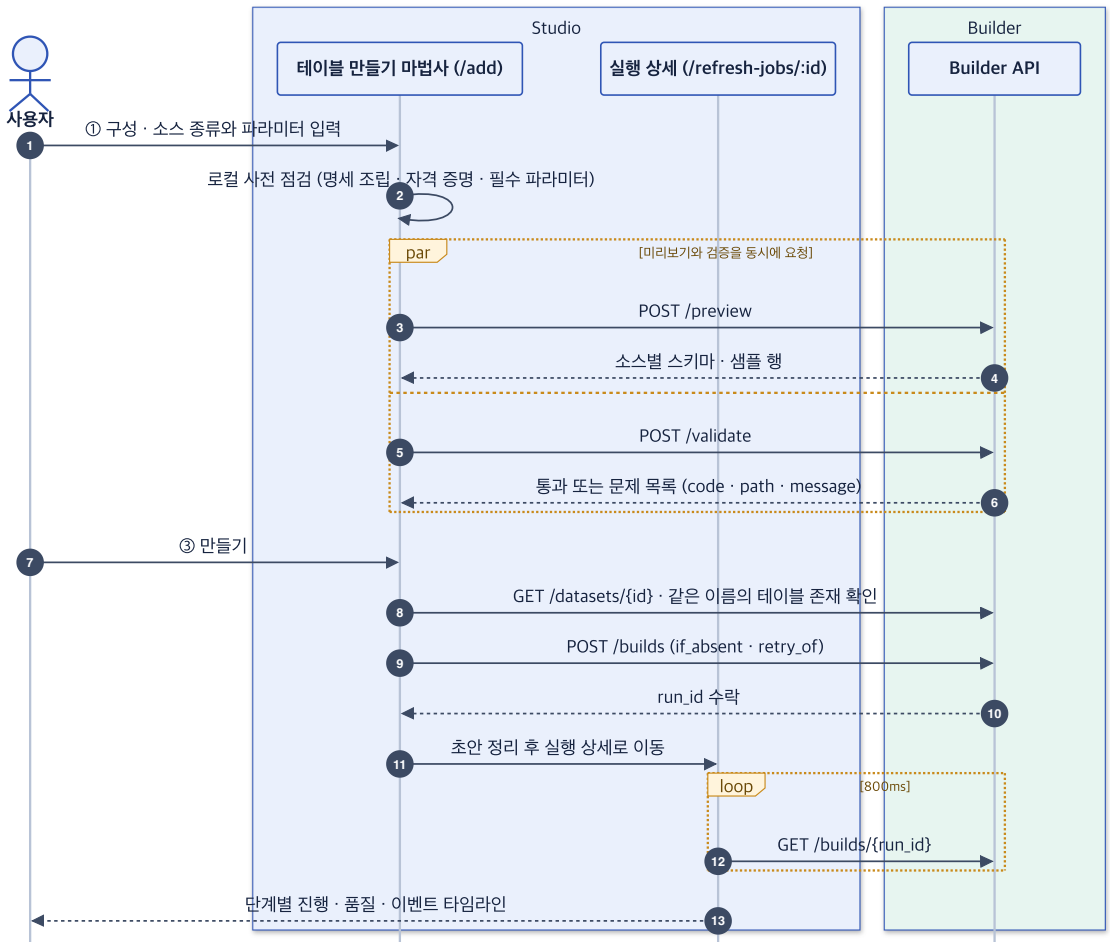
- BuildSpec은 실행 입력을 선언하는 계약이며 Python dataclass로 정의함
- BuildManifest는 실행마다 한 건씩 남는 결과 기록임
- 관계형 스키마가 아니므로 UML 클래스 다이어그램 표기법으로 정리함

엔티티 관계도 - 저장 구조 (ERD)



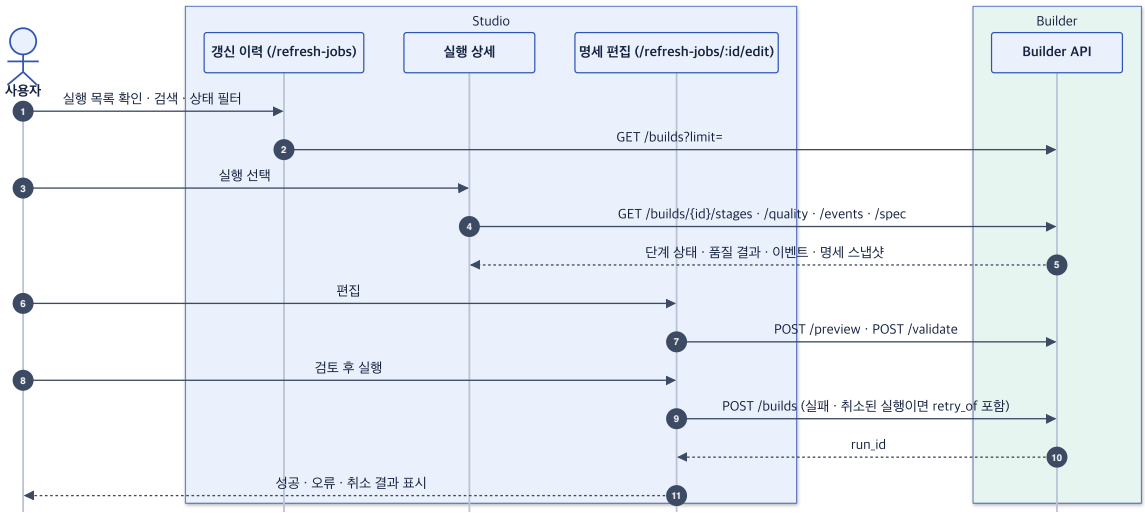
- 상태는 SQLite와 로컬 파일에만 보관함
- `builds`는 manifest.json과 종료 기록에서 다시 만들 수 있는 파생 인덱스임
- `tables·table_snapshots`는 웨어하우스 카탈로그이며 스냅샷 본문은 Parquet 파일임
- 주요 컬럼만 표시함

기능 처리도 - 테이블 만들기



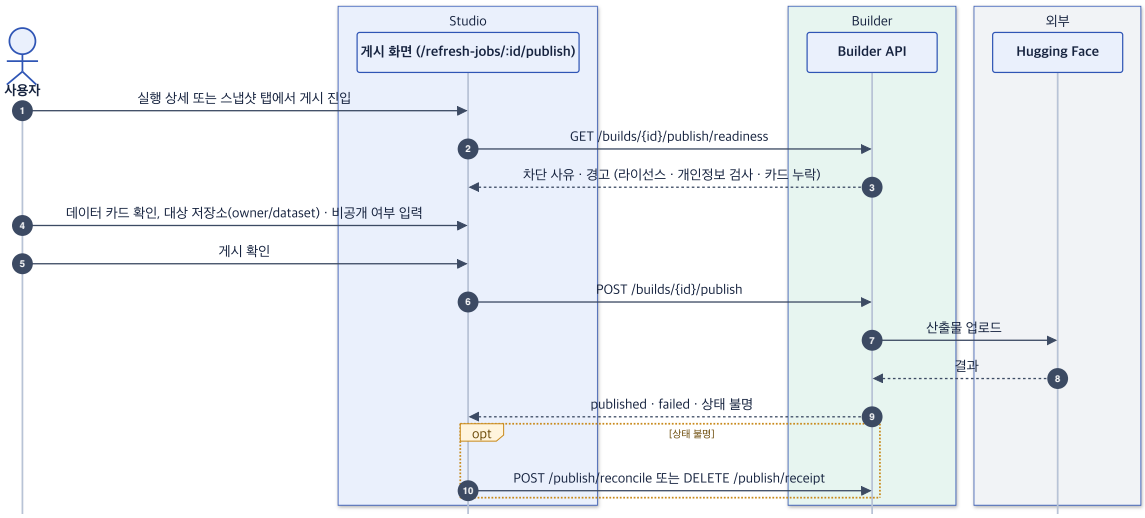
- 사용자 조작(①⑦)은 Studio 안에서 처리되고, Builder API 호출은 미리보기·검증·존재 확인·제출 네 곳임
- 미리보기와 검증은 동시에 요청하며, 늦게 도착한 이전 응답은 버림
- 제출 직전에 같은 이름의 테이블이 있는지 다시 확인하고, 결과가 달라졌으면 제출하지 않음
- 마법사는 완료를 기다리지 않고 실행 상세로 넘어가며, 진행은 실행 상세가 따라감

기능 처리도 - 실행 검토·재실행



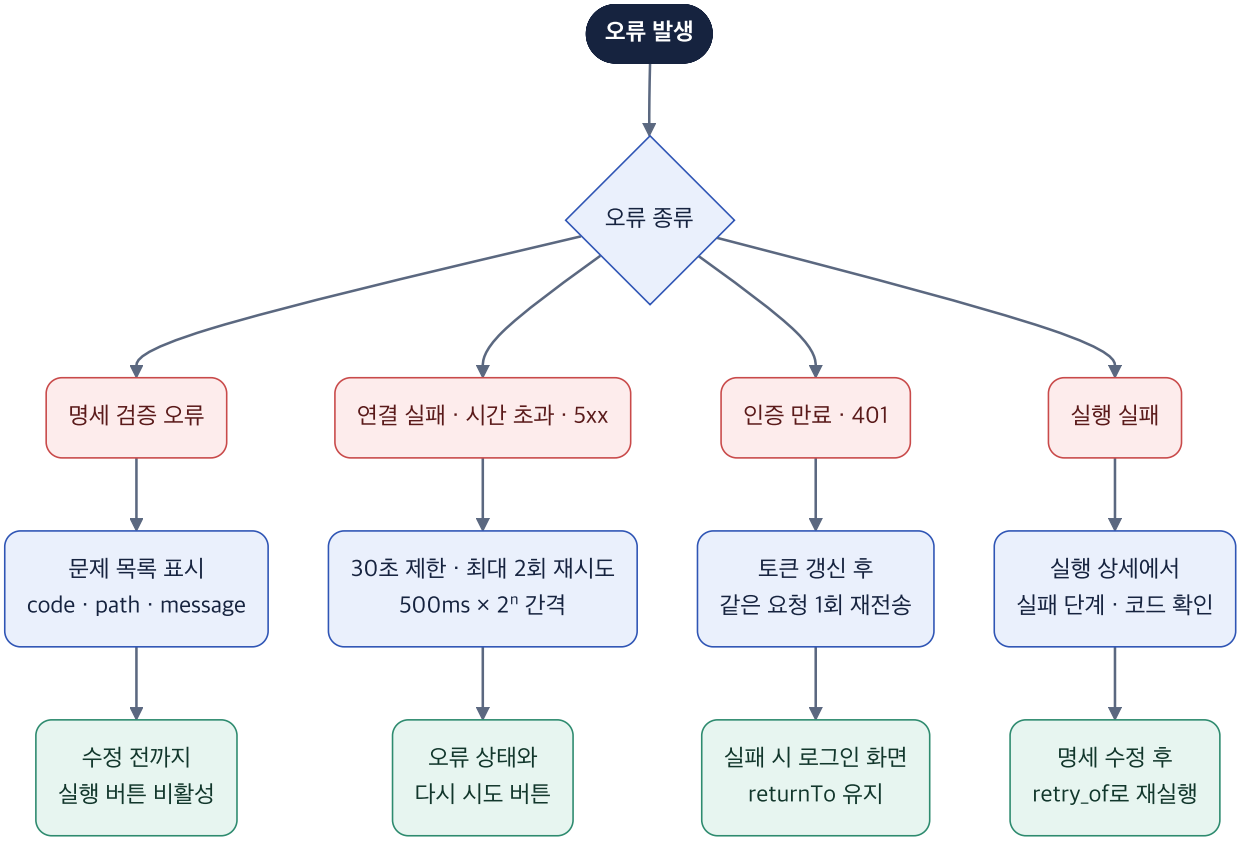
- 실행 상세는 단계·품질·이벤트·명세를 따로 조회해 한 영역의 실패가 전체를 막지 않게 함
- 실패하거나 취소된 실행을 다시 돌릴 때만 `retry_of`를 보내며, 수집 체크포인트를 이어받음
- 성공한 실행을 다시 돌리는 것은 갱신이며 새 스냅샷을 만듦

기능 처리도 - 게시



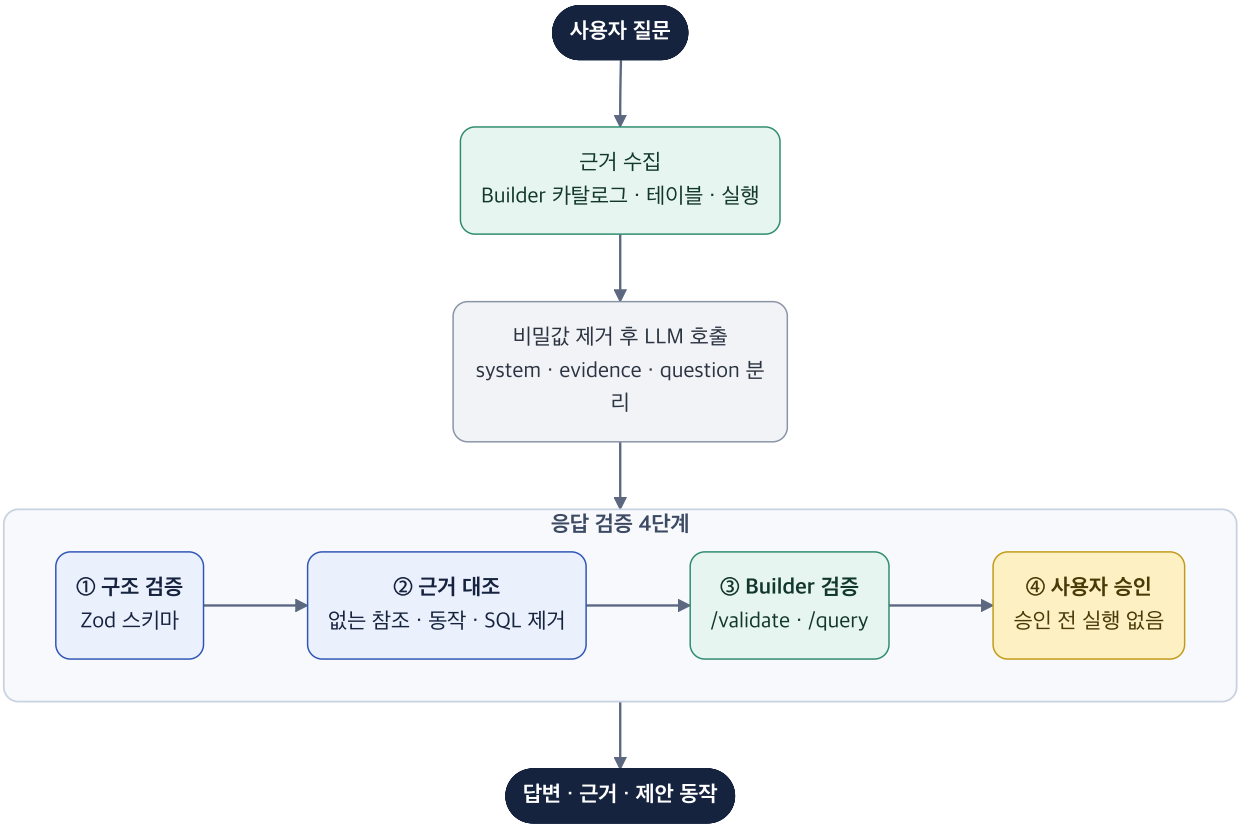
- 게시 전 준비 상태를 조회해 라이선스, 개인정보 검사, 카드 누락 등 차단 사유를 먼저 표시
- Studio의 게시 대상은 Hugging Face이며, Builder CLI는 로컬·Kaggle 게시도 지원
- 게시 요청은 자동 재시도하지 않으며, 결과가 불명확하면 대조 또는 영수증 초기화로 복구

기능 처리도 - 오류 처리



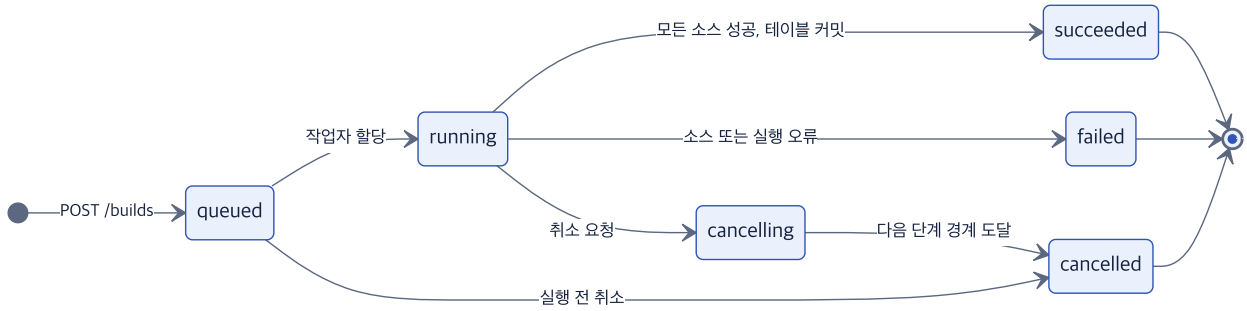
- ① 명세 검증 오류 — 문제 목록을 표시하고 수정 전까지 실행을 막음
- ② 연결 실패 — 30초 제한과 최대 2회 재시도 후 오류 상태와 다시 시도 버튼 표시. 미리보기·제출·게시·쿼리는 재시도하지 않음
- ③ 인증 만료 — 토큰을 갱신해 같은 요청을 한 번 다시 보내고, 실패하면 로그인 화면으로 이동
- ④ 실행 실패 — 실행 상세에서 실패 단계와 코드를 확인하고 명세를 고쳐 재실행
- 오프라인 모드나 자동 재연결은 두지 않음. 주기적 조회가 돌아오면 화면이 갱신됨

기능 처리도 - Ask KPubData 응답 처리



- 프롬프트 역할 분리: 고정 지시(system), 신뢰하지 않는 근거(evidence), 사용자 질문을 구분해 전달
- 전송 전 비밀값 제거: 키·토큰 이름의 필드, `key=value` 형태, 무작위성이 높은 긴 문자열을 자리표시자로 치환
- 4단계 검증: ① Zod 구조 검증 → ② 근거 대조 → ③ Builder 검증 → ④ 사용자 승인
- SQL 안전성 검사는 Studio가 아니라 Builder의 읽기 전용 검사가 담당
- 허용 동작 6종: OPEN_PROVIDER, OPEN_BUILD, OPEN_QUALITY, PATCH_BUILDSPEC, CREATE_BUILD_DRAFT, ADD_REPORT_BLOCK
- 실행 시작, 게시, 자격 증명 변경, SQL 자동 실행은 제안할 수 없음
- LLM 키는 사용자가 직접 입력(BYOK)하며 기본적으로 메모리에만 보관

기능 처리도 - 비동기 실행 상태 전이



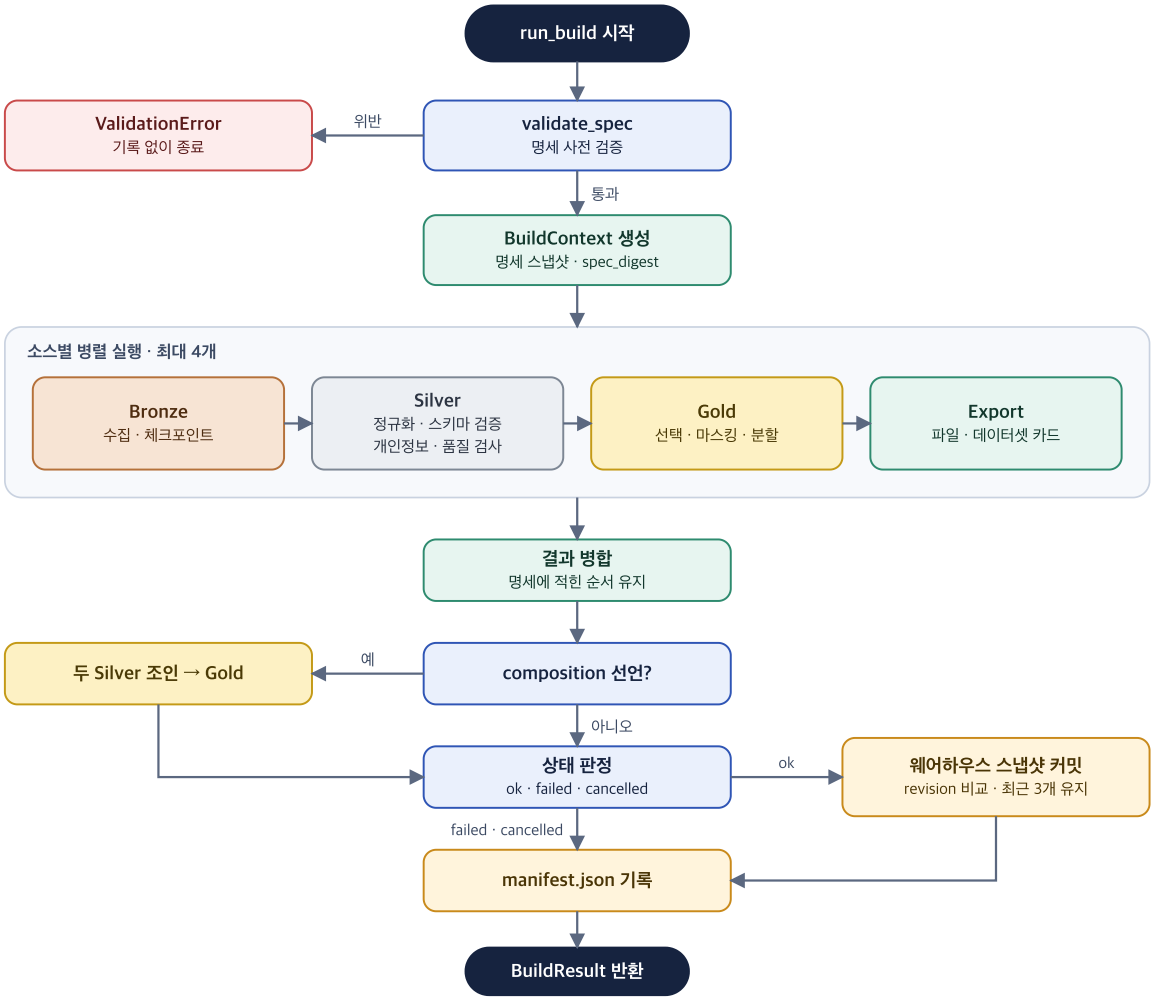
상태	의미
`queued`	대기열에 들어가 실행을 기다림
`running`	소스를 수집하고 단계를 수행하는 중
`cancelling`	취소가 요청되어 다음 단계 경계에서 끝나기를 기다림
`succeeded`	모든 소스가 성공하고 테이블 커밋까지 끝남
`failed`	소스 실행, 조립 또는 테이블 커밋에서 실패
`cancelled`	사용자 요청으로 취소됨

취소 규칙

요청 시점 상태	결과	HTTP
`queued`	즉시 cancelled. 파이프라인을 한 번도 실행하지 않음	200
`running`	cancelling으로 바뀌고 다음 단계 경계에서 cancelled	200
`cancelling` · `cancelled`	변화 없음 (여러 번 요청해도 같은 결과)	200
`succeeded` · `failed`	취소 불가	409

- 취소는 협력적으로 처리함. 작업 스레드를 강제로 끊지 않고 Bronze·Silver·Gold·Export 사이의 경계에서만 확인
- 마지막 경계를 지나 정상 종료가 확정되면 그 뒤의 취소 요청은 409로 거절
- 취소된 실행이 이미 만든 산출물은 `partial=true`인 Manifest와 함께 보존
- 실행 제한 기본값: 동시 실행 2건, 사용자별 진행 중 2건, 대기 10건, 실행 시간 6시간
- Studio는 종료 상태가 될 때까지 800ms 간격으로 `GET /builds/{id}`를 조회

알고리즘 명세서


run_build 시작
validate_spec
명세 사전 검증
위반
ValidationError
기록 없이 종료
통과
BuildContext 생성
명세 스냅샷 · spec_digest
소스별 병렬 실행 · 최대 4개
Bronze
수집 · 체크포인트
Silver
정규화 · 스키마 검증
개인정보 · 품질 검사
Gold
선택 · 마스킹 · 분할
Export
파일 · 데이터셋 카드
결과 병합
명세에 적힌 순서 유지
composition 선언?
예
두 Silver 조인 → Gold
아니오
상태 판정
ok · failed · cancelled
ok
웨어하우스 스냅샷 커밋
revision 비교 · 최근 3개 유지
failed · cancelled
manifest.json 기록
BuildResult 반환

알고리즘 명세서

- ① `run_build()`가 BuildSpec을 받아 `validate_spec()`으로 먼저 검증. 위반이 있으면 `ValidationError`로 끝나며 작업공간도 기록도 만들지 않음
- ② `BuildContext.create()`로 실행 작업공간을 만들고 명세 스냅샷(buildspec.yaml)과 `spec_digest` 기록
- ③ 소스를 스레드 풀로 병렬 실행 (최대 4개). 결과는 명세에 적힌 순서대로 모음
- ④ 소스마다 Bronze → Silver → Gold → Export 순으로 진행. 단계 경계마다 취소 여부 확인
- ⑤ Silver에서 스키마 검증, 개인정보 검사, 품질 검사를 통과해야 Gold로 넘어감
- ⑥ composition을 선언했으면 두 소스의 Silver를 조인해 하나의 Gold 생성
- ⑦ 상태 판정: 취소되었으면 cancelled, 오류가 하나라도 있으면 failed, 아니면 ok
- ⑧ 상태가 ok일 때만 Gold를 웨어하우스 스냅샷으로 커밋. 시작 시점의 revision과 비교해 충돌하면 거부
- ⑨ manifest.json을 기록하고 `BuildResult(status, outcomes, manifest_path, …)` 반환

상태값

계층	값	비고
오케스트레이터·Manifest	`ok` · `failed` · `cancelled`	실행 결과의 원본 값
HTTP API 작업 상태	`queued` · `running` · `cancelling` · `succeeded` · `failed` · `cancelled`	`ok`는 `succeeded`로 표시. 테이블 커밋이 실패하면 `failed`
품질 검사	`pass` · `warn` · `fail`	평가하지 못한 규칙은 결과에서 제외

Manifest가 남지 않는 경우

- 명세 검증에서 거부된 요청: 실행 자체가 시작되지 않음
- 대기 중 취소, 서버 재시작 등으로 파이프라인에 들어가기 전에 끝난 비동기 실행: `run_endings` 표에 종료 기록으로 남김
- 파이프라인에 들어간 뒤 취소된 실행은 `status=cancelled`, `partial=true`로 Manifest를 남김

알고리즘 명세서 - Silver 정규화

○ 명세에 선언한 규칙만 아래 순서로 적용하며, 선언하지 않은 변환은 하지 않음

순서	규칙	명세 항목	동작
1	읽기 형식	`read_as`	지정한 컬럼을 문자열로 읽음 (앞자리 0 보존 등)
2	결측 토큰	`null_tokens`, `column_null_tokens`	'-', 'N/A' 같은 값을 결측으로 처리. 컬럼별 토큰은 전역 토큰에 더해짐
3	컬럼 병합	`coalesce`	이름만 다른 여러 컬럼을 하나로 합침. 값이 서로 다르면 실패
4	이름 변경	`rename`	원본 필드명을 표준 이름으로 변경
5	자리 채움	`zfill`	지정한 자릿수까지 왼쪽을 0으로 채움 (행정 코드 등)
6	형 변환	`casts`	bool·date·datetime·float·int·str과 천 단위 쉼표 숫자(int_comma, float_comma). 변환으로 값이 사라지면 오류
7	파생 컬럼	`derived`	date_parts(연·월·일 3개 컬럼 → 날짜), join_key(여러 컬럼 → 조인 키)

- 정규화는 DuckDB SQL로 수행하고 결과는 `silver/{source}/table.parquet`에 저장
- Silver는 행과 컬럼을 모두 보존. 선택과 필터는 Gold에서만 수행
- 필수 컬럼과 자료형 검증, 통계(`stats.json`), 미리보기(`preview.json`) 생성

품질 검사 규칙

규칙	검사 내용	기준
`required_column`	필수 컬럼 존재 여부	스키마 계약의 required
`dtype`	컬럼 자료형 일치 여부	스키마 계약의 dtypes
`max_duplicate_rate`	중복 행 비율	0~1 사이 상한
`max_null_ratio`	컬럼별 결측 비율	컬럼별 상한
`min_rows`	최소 행 수	하한
`range`	숫자 컬럼의 값 범위	min·max (경계 포함)
`compare_columns`	두 컬럼의 대소·동일 관계	eq·ne·gt·gte·lt·lte

- 결과는 `pass`·`warn`·`fail`. 규칙마다 심각도(warn·fail)를 명세에서 지정
- fail이 하나라도 있으면 해당 소스를 Gold 전에 중단, warn은 기록하고 진행
- 평가하지 못한 규칙은 통과로 치지 않고 결과에서 뺌
- 이전 실행 대비 컬럼 추가·삭제, 자료형 변경, 행 수 급변은 스키마 변화로 기록만 하고 중단하지 않음

알고리즘 명세서 - 개인정보 검사와 분할

개인정보 검사

종류	검출 방법
주민등록번호	하이픈이 있는 형식, 또는 13자리 숫자에 대해 날짜와 검증 숫자 확인
전화번호	국내 전화번호 형식 정규식
이메일	이메일 형식 정규식
사업자등록번호	사업자등록번호 형식 정규식
컬럼명	성명·주소·연락처 등 개인정보를 뜻하는 이름 포함 여부

- 문자열 컬럼의 서로 다른 값을 대상으로 검사
- 정책(`pii.mode`): `block`은 검출 시 소스 중단, `warn`은 기록 후 진행, `allow`는 통과 (게시와 함께 쓸 수 없음)
- 마스킹은 선언 기반: kpubdata가 선언한 개인정보 컬럼과 명세의 `gold.pii_columns`에 적용
- 문자열은 `[masked]`로, 그 밖의 자료형은 결측으로 바꿈
- `publish_unmasked`로 예외를 둔 컬럼은 Manifest에 경고로 남음

분할

- `ratio`: 시드와 행 순번의 해시로 정렬한 뒤 비율대로 자름 (hash-sort-v2). 같은 입력이면 항상 같은 결과이고 개수가 정확히 맞음
- `key`: 지정한 컬럼의 값별로 나눔. 결측은 `__null__`, 컬럼 없음은 `__missing__`
- 분할 이름은 자유롭게 지정 (train·valid·test는 관례)
- 비율 분할에서 시간 컬럼을 무작위로 나누는 경우는 명세 검증에서 걸러냄 (time_column_random_split)
- 결과는 `gold/{name}/splits/{split}.parquet`에 저장

알고리즘 명세서 - 웨어하우스 커밋

○ 상태가 ok인 실행의 Gold를 읽기 전용 스냅샷으로 커밋하고 테이블의 현재 스냅샷을 바꿈

- 스냅샷은 임시 위치에 쓴 뒤 검증을 거쳐 `tables/{id}/snapshots/{id}/`로 옮기고 읽기 전용으로 만듦
- 테이블마다 revision을 두고, 커밋할 때 revision을 하나 올림
- 이전 스냅샷은 최근 3개까지 보관하고 나머지는 정리. 저장된 분석이 붙잡고 있는 스냅샷은 정리하지 않음

커밋 거부 사유

사유 코드	발생 조건	처리
`table_exists`	새 테이블로 만들기를 요청했는데 같은 이름의 테이블이 이미 있음	커밋하지 않고 실패로 기록
`empty_result`	결과가 0행이고 `allow_empty`가 꺼져 있음	기존 스냅샷을 유지
`conflict`	실행 시작 때의 revision과 현재 revision이 다름	먼저 커밋한 쪽을 유지하고 거부
`commit_failed`	파일 이동·카탈로그 기록 중 오류	실패로 기록, Manifest에 사유 남김

- 커밋은 모든 소스가 끝난 뒤 한 스레드에서 수행. 실행 시작 때 기억한 revision과 비교하는 방식
- 커밋을 Manifest 기록보다 먼저 수행해, 커밋 실패가 Manifest와 인덱스에 남도록 함

알고리즘 명세서 - 읽기 전용 SQL 검사와 비밀값 제거

읽기 전용 SQL 검사 (Builder)

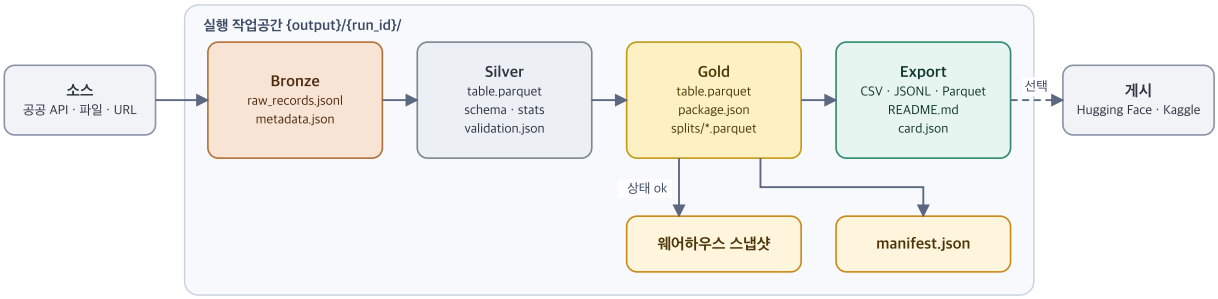
- 입력은 비어 있지 않고 64KiB 이하
- sqlglot으로 DuckDB 문법 분석. 구문 오류는 거부
- 문장은 정확히 하나, SELECT·CTE·집합 연산만 허용 (DDL·DML 불가)
- 재귀 CTE 불가, `dataset`이라는 이름의 CTE 불가
- `VALUES`, `UNNEST`, `LATERAL`, 테이블 함수, 표본 추출(`TABLESAMPLE`) 불가
- 환경을 들여다보는 함수(`current_setting`, `getenv`, `version` 등 44종) 불가
- 결과가 매번 달라지는 함수(`random`, `uuid`, `now` 등 18종) 불가
- 실제 테이블 참조는 따옴표·스키마 지정 없이 `dataset` 하나만 허용
- 검사를 통과한 문장은 주석을 뺀 표준형으로 다시 만들어 실행하며, 원문은 DuckDB에 전달하지 않음
- 검사는 3중 방어의 첫 단계. 그 뒤에 잠긴 DuckDB 연결과 별도 자식 프로세스가 있음

LLM 전송 전 비밀값 제거 (Studio)

- 필드 이름이 키·토큰·시크릿을 뜻하면 값 제거 (`servicekey`, `api_key`, `*token`, `*secret`)
- 본문 속 `key: value`, `key=value` 형태 제거
- 24자 이상이고 무작위성(섀넌 엔트로피 4.0비트/문자 이상)이 높은 문자열 제거
- 제거한 값은 자리표시자로 바꾸고 응답을 받은 뒤 브라우저에서만 되돌림
- Builder가 확인해 준 실행 ID와 근거 ID는 제거 대상에서 제외

데이터 수집처리 정의서

○ Medallion 구조 기반 수집·처리 — Bronze / Silver / Gold 3단계 + Export



단계	처리 내용	사용 기술	입력 → 출력
Bronze (원천 수집)	공공 API·업로드 파일·URL의 원본을 그대로 수집. 수집 시각·파라미터·건수·체크섬 기록. 파라미터 조합별 체크포인트	kpubdata, URL 수집기	외부 원본 → `bronze/{source}/raw_records.jsonl`, `metadata.json`
Silver (정제·검증)	명세에 선언한 규칙만 적용: 결측 토큰 → 컬럼 병합 → 이름 변경 → 자리 채움 → 형 변환 → 파생 컬럼. 필수 컬럼·자료형 검증, 통계·미리보기 생성, 개인정보·품질 검사	DuckDB	Bronze → `silver/{source}/table.parquet`, `schema.json`, `stats.json`, `validation.json`
Gold (패키징)	행 필터와 컬럼 선택, 선언한 개인정보 컬럼 마스킹, 분할, 두 소스 조인	DuckDB	Silver → `gold/{name}/table.parquet`, `package.json`, `splits/*.parquet`
Export (내보내기)	Gold를 exports 설정에 맞춰 파일로 저장하고 데이터셋 카드 생성	Exporter 6종 (csv, jsonl, parquet, markdown, huggingface, kaggle)	Gold → 산출물 파일, `README.md`, `card.json`
웨어하우스 커밋	상태가 ok인 실행의 Gold를 읽기 전용 스냅샷으로 커밋. 최근 3개 유지	SQLite 카탈로그, Parquet	Gold → `tables/{id}/snapshots/{id}/`

- 주요 선언 항목: 수집 파라미터(params·param_grid), 정규화 규칙(schema), 품질 기준(quality), 분할(splits), 내보내기(exports)
- Silver는 행과 컬럼을 모두 보존하고, 선택과 필터는 Gold에서만 수행

데이터 수집처리 정의서 - 소스 종류와 제한

소스 종류	입력	형식	제한·규칙
public_api	kpubdata 제공 기관과 데이터셋, 수집 파라미터	kpubdata 표준 레코드	페이지 단위로 수집해 바로 저장. `param_grid` 조합별 체크포인트. 제공 기관별 분당 호출 상한 설정 가능
file	`POST /uploads`로 올린 파일의 `upload_id`	csv, json, jsonl, parquet	`upload_id` 형식 검사, 인코딩 지정 가능 (기본 utf-8)
url	HTTPS 주소	json, jsonl, csv	GET만 허용, 인증 헤더 불가, 주소에 사용자 정보 불가. 연결 5초·읽기 15초, 리다이렉트 5회, 20MiB 이하. DNS 고정으로 내부망 접근 차단

수집 기록 (provenance)

- 소스마다 수집 시각, 파라미터, 레코드 수, 체크섬을 `metadata.json`과 Manifest에 기록
- 재실행(`retry_of`)은 하루 이내의 `param_grid` 체크포인트를 이어받아 끝난 조합을 다시 받지 않음
- 이어받지 못해 처음부터 다시 받은 소스는 Manifest의 `checkpoints_not_reused`에 남음

정기 수집 일정 (GitHub Actions)

분야	주기	상태
기상 예보	3시간마다	운영
부동산 실거래 (매매·전월세)	매월 1~10일	운영
관광 정보	매주 월요일	운영
의약품 안전 사용(DUR)	매주 토요일	운영
대기질	수동 실행	일정 중지

- 각 일정은 공통 게시 작업(`publish-dataset.yml`)을 호출해 Hugging Face에 게시
- 대기질은 이용 조건 문제로 일정을 내리고 수동 실행만 남김
- 게시가 실패하면 이슈를 자동으로 열거나 갱신
- 신규 데이터셋 탐색과 활용 신청 승인 확인도 매일 한 번 실행

프로그램 목록 - KPubData Builder

기능ID	패키지·모듈	역할
A02_B01~02	spec/	BuildSpec 모델, YAML 로더, 검증기, 실행 스냅샷, 지문, 갱신 주기, param_grid
A02·A03	pipeline/	오케스트레이터(run_build), 실행 문맥, 취소 계약, 미리보기, 내보내기, 실패 분류, 카드 자료
A01_B01	stages/bronze/	소스 수집, 임시 기록, 체크포인트, 저장
A01_B02	ingestion/	URL 수집(내부망 차단), 파일 읽기(csv·json·jsonl·parquet)
A02_B05	stages/silver/, tabular/	DuckDB 실행 환경, 적재, 정규화, 검증, 통계, 미리보기, 스키마 변화 감지
A02_B06	quality/	품질 평가기와 결과 모델
A06_B01	stages/silver/pii.py, stages/gold/pii.py	개인정보 검사, 선언 컬럼 마스킹
A02_B07	stages/gold/	선택, 분할, 조인, 패키지 구성, 저장
A03_B01	stages/gold/card.py, catalog_info.py	데이터셋 카드(README.md, card.json) 생성
A03_B02	exporters/	csv, jsonl, parquet, markdown, huggingface, kaggle와 등록 체계
A03_B05	publishers/	local, huggingface, kaggle
A03_B03	manifest/, events/	Manifest 모델·기록·출처·체크섬, 실행 이벤트와 종료 기록
A03_B04	warehouse/	테이블 카탈로그(SQLite), 스냅샷 배치, 커밋, 정리, 백업
A06_B03	query/	읽기 전용 SQL 검사, 격리 실행, 집계, 행 조회, 프로파일, 내보내기
A04·A08	service/	HTTP 서버, 경로, 인증, 비동기 작업, 게시, 모니터링, 관리, 재배포 판정
A06_B04	credentials/, uploads/	자격 증명 암호화 저장, 업로드 저장
A01_B04	agent/, verify/	신규 데이터셋 탐색, 활용 신청 승인 확인, 명세 실검증
-	store/	실행 인덱스, 상태 목록, 스키마 버전, 상태 백업·잠금
-	egress.py, rate_limit.py	제공 기관별 호출 상한, 분당 요청 제한
-	retention.py, replay.py, diff.py	실행 보존 기간 정리, 고정 자료 재생, Manifest 비교
-	settings_catalog.py, logging_redaction.py	환경변수 목록, 로그에서 키 제거
-	cli.py	명령줄 진입점 (kpubdata-builder)

프로그램 목록 - Builder CLI

명령	용도
`validate`	BuildSpec YAML 검증
`preview`	산출물을 쓰지 않고 스키마와 샘플 행 확인
`build`	Medallion 파이프라인으로 BuildSpec 실행
`publish`	산출물을 로컬 또는 원격 대상에 게시
`serve`	Builder HTTP 서비스 실행
`settings`	실제 적용 중인 설정 출력 (비밀값 가림)과 점검
`fixtures export`	재생용 고정 자료를 디렉터리로 복사
`rebuild-index`	파일을 훑어 실행 인덱스 재구성
`discover` · `monitor` · `pipeline`	data.go.kr 메타데이터 탐색, 활용 신청 승인 확인, 신규 데이터셋 온보딩 자동화
`verify`	데이터셋 명세를 실제 API로 검증
`prune-cancelled` · `prune-runs`	취소된 부분 실행, 보존 기간이 지난 실행 정리
`delete-user`	사용자와 그 데이터를 단계별로 삭제
`warehouse-gc` · `warehouse-hold`	필요 없는 스냅샷 정리, 스냅샷 보존 지정
`warehouse-backup` · `warehouse-restore`	테이블 카탈로그와 스냅샷 파일 백업·복원
`backup` · `verify-backup` · `restore`	전체 상태 백업, 백업 검증, 복원

- 실행 파일 이름은 `kpubdata-builder`
- `validate`·`preview`·`build`는 HTTP 서비스와 같은 파이프라인을 호출

프로그램 목록 - KPubData Studio

기능ID	디렉터리 (src/features)	역할
A04_B01	`home`, `onboarding`	홈 데이터(조치 필요, 연결 문제, 최근 스냅샷), 첫 실행 체크리스트, 활용 신청 안내
A04_B02	`discover`	카탈로그 조회, 평탄화·필터, 비교 표
A04_B02	`add-data`	테이블 만들기 마법사 3단계, 초안 저장, 기존 테이블 확인, 자격 증명 선행 조건, 업로드
A04_B02	`preview`, `validation`	`/preview`, `/validate` 클라이언트
A04_B02	`build-spec`	Studio↔Builder 명세 변환, YAML, 실행별 명세 보관, 초안, 비교, 개정 이력, 편집 6단계
A04_B03	`datasets`, `data-table`	테이블 목록 모델, 실행 이력, 웨어하우스 보기, 프로파일, 품질 추세, 공통 표, 행 조회
A04_B04	`sql`, `charts`, `export`	SQL 실행, 저장된 분석, 테이블 탐색기, 집계 차트, 내보내기
A04_B05	`reports`, `workspace`	리포트 저장소·근거·내보내기·편집기, 최근 작업, 저장한 BuildSpec
A04_B06	`runs`, `artifacts`	비동기 실행과 상태 조회, 이벤트 조회, 실행 상세, 산출물
A04_B06	`publish`	게시 클라이언트, 데이터 카드 미리보기, 복구 패널
A04_B07	`provider`, `version-check`	연결 표, 키 확인, Builder 버전·계약 비교, 불일치 배너
A06_B02	`licence`	이용 조건 요약, 실행별 라이선스
A06_B04	`auth`	Keycloak 연동, 토큰 갱신, 로그인 보호, 사용자별 저장소 분리
A07_B01	`assistant`	Ask KPubData: 서랍·본문, 세션, 키 설정, 프롬프트, 구조 검증, 근거 대조, 동작, 질의, 비밀값 제거
A08_B01	`monitoring`, `admin`	모니터링 모델과 패널, 관리자 판별과 관리 화면
A08_B02	`quality`	품질 모델(PASS·WARN·FAIL·N/A), 항목 표, 배지

- 기능 폴더마다 Builder용과 데모용 클라이언트를 두고 같은 계약 테스트로 검증
- Builder API 호출은 `shared/lib/builderApi.ts` 한 곳에서 수행

프로그램 목록 - Studio 화면과 개발 스택

파일 (src/pages)	화면	경로
`LoginPage` · `SignupPage`	로그인, 가입	`/login`, `/signup`
`HomePage`	홈	`/`
`DiscoverPage`	카탈로그	`/discover`
`AddDataPage`	테이블 만들기	`/add`
`DatasetCatalogPage` · `DatasetDetailPage`	테이블, 테이블 상세	`/tables`, `/tables/:id`
`SqlWorkspacePage` · `AnalysesPage`	SQL 작업 공간, 저장된 분석	`/sql`, `/analyses`
`WorkspacePage`	작업대	`/workspace`
`ReportsPage` · `ReportEditorPage`	리포트 목록, 편집기	`/reports`, `/reports/:id`
`BuildsPage`	갱신 이력, 실행 상세	`/refresh-jobs`, `/refresh-jobs/:id`
`NewBuildPage`	명세 편집·재실행	`/refresh-jobs/:id/edit`
`BuildArtifactsPage` · `BuildPublishPage`	산출물, 게시	`…/artifacts`, `…/publish`
`QualityPage` · `MonitoringPage`	품질, 모니터링	`/quality`, `/monitoring`
`ProviderPage` · `SettingsPage`	연결, 설정	`/connections`, `/settings`
`AdminPage` · `AssistantPage`	관리, Ask KPubData	`/admin`, `/assistant`

개발 스택

구분	내용
Builder	Python 3.10~3.13, DuckDB 1.2 이상, pyarrow, SQLite, sqlglot, uv
Studio	React 19.3, TypeScript 6, Vite 8, React Router 7, Zustand 5, Tailwind CSS 4, Zod 4, react-hook-form, yaml
다국어	i18next, 한국어(기본)·영어
인증	Keycloak, keycloak-js (OIDC Authorization Code + PKCE)
실행 환경	Node 22.13 이상, Docker (Studio는 nginx 이미지로 제공)

테이블 정의서 - BuildSpec

○ BuildSpec 필드 구조 (dataset_id / title / description / sources[] 는 필수)

필수/선택	필드명	타입	설명
필수	`dataset_id`	string	데이터셋 식별자
필수	`title`	string	사람이 읽는 제목
필수	`description`	string	목적과 데이터 설명
필수	`sources[]`	array<SourceRef>	입력 소스 (1개 이상)
선택	`exports[]`	array<ExportTarget>	내보내기 대상. 비워 두면 테이블 커밋까지만 수행
선택	`splits`	SplitSpec	분할 정의. 없으면 분할하지 않음
선택	`quality`	QualityPolicy	품질 기준. 없으면 검사 생략
선택	`pii`	PiiPolicy	개인정보 검사 정책. 없으면 검사 생략
선택	`composition`	CompositionSpec	두 소스를 조인해 하나의 Gold 생성
선택	`license`	string	SPDX 식별자 또는 자유 기재. 게시할 때 필수
선택	`license_name` · `license_link`	string	license가 other일 때의 이름과 조건 링크
선택	`attribution`	string	출처 표기 문구. 카드의 출처 항목에 표시
선택	`publish`	boolean	실행 후 게시 여부 (기본 false)
선택	`refresh_cadence`	string	갱신 주기 (ISO-8601 기간: P1D, PT6H, P1W). 갱신 지연 판정에 사용
선택	`metadata`	object	산출물에 실을 임의 메타데이터

- ※ DB 테이블이 아닌 실행 계약의 필드 구조를 테이블 정의서 형식으로 정리함 (출처: `spec/models.py`, `spec/loader.py`)
- 이전 형식의 `normalization_mode`, `transforms`는 없어졌으며 넣으면 로더가 거부함

테이블 정의서 - SourceRef · SchemaContract

○ SourceRef — BuildSpec의 sources[] 항목

필드명	타입 (기본값)	설명
kind	string (public_api)	소스 종류: public_api | file | url
provider · dataset	string	kpubdata 제공 기관과 데이터셋. public_api에서 필수
params	object	수집 파라미터
param_grid	object	파라미터 축. 조합마다 따로 수집
alias	string	출력 키 이름. 조인에서 참조
schema	SchemaContract	Silver 정규화·검증 계약
gold	GoldSelection	Gold에 남길 컬럼과 행. 없으면 Silver 전체
upload_id · format · encoding	string (encoding: utf-8)	파일 소스: 업로드 식별자, 형식, 인코딩
endpoint · method	string (method: GET)	URL 소스: 주소, 메서드
allow_empty	boolean (false)	0행 결과가 기존 테이블을 대체하도록 허용

○ SchemaContract — 소스별 정규화·검증 계약 (모두 선택)

필드명	타입	설명
required	tuple<string>	필수 컬럼
dtypes	dict	컬럼별 기대 자료형
casts	dict	컬럼별 형 변환
rename	dict	원본 필드명 → 표준 이름
derived	tuple<DerivedColumn>	파생 컬럼 (name, kind, columns)
read_as	dict	읽기 형식 지정 (str만 허용)
null_tokens · column_null_tokens	tuple · dict	전역·컬럼별 결측 토큰
coalesce	dict	여러 컬럼을 하나로 병합
zfill	dict	왼쪽 0 채움 자릿수

- ※ 출처: spec/models.py

테이블 정의서 - Gold · 내보내기 · 분할 · 개인정보

모델	필드명	타입	설명
GoldSelection	`select`	tuple<string>	남길 컬럼과 순서. 비우면 전체
	`filters`	tuple<GoldFilter>	행 필터. select보다 먼저 적용
	`pii_columns`	tuple<string>	개인정보로 선언하는 컬럼
	`publish_unmasked`	tuple<string>	마스킹 없이 게시할 선언 컬럼 (Manifest 경고)
GoldFilter	`column` · `op` · `value`	string · string · any	연산: eq, ne, gt, ge, lt, le, in, not_null
ExportTarget	`kind` · `output_path` · `options`	string · string · object	내보내기 형식, 경로, 형식별 옵션
SplitSpec	`mode` · `ratios` · `key` · `seed`	string · dict · string · int	ratio 또는 key 분할
PiiPolicy	`mode` · `allow_columns`	string · tuple	block(기본) | warn | allow, 검사 예외 컬럼

- ※ 출처: `spec/models.py`, `spec/loader.py`

테이블 정의서 - 품질 · 조인

모델	필드명	타입 (기본값)	설명
QualityPolicy	`max_duplicate_rate` (+ severity)	float? (warn)	중복 행 비율 상한
	`max_null_ratio` (+ severity)	dict	컬럼별 결측 비율 상한
	`min_rows` (+ severity)	int? (warn)	최소 행 수
	`range`	tuple<RangeRule>	값 범위 규칙
	`compare_columns`	tuple<CompareColumnsRule>	컬럼 비교 규칙
RangeRule	`column` · `min` · `max` · `severity`	string · float? · float? · string	경계 포함 범위
CompareColumnsRule	`left` · `operator` · `right` · `severity`	string × 4	연산: eq, ne, gt, gte, lt, lte
CompositionSpec	`name` · `join`	string · JoinSpec	조인 결과의 이름, 조인 정의
JoinSpec	`left` · `right`	string	조인할 두 소스의 alias
	`left_key` · `right_key` 또는 `keys`	string · tuple	단일 키 축약형 또는 복합 키 쌍 (둘 중 하나)
	`type`	string (inner)	inner | left
	`cardinality`	string?	one_to_one | one_to_many | many_to_one | many_to_many
	`on_duplicate_key` · `on_null_key`	string (warn)	키 중복·결측 시 warn | fail

- 조인은 두 소스의 등가 조인 하나만 지원
- 심각도는 warn 또는 fail
- ※ 출처: `spec/models.py`, `spec/loader.py`

테이블 정의서 - BuildManifest

○ BuildManifest 필드 구조 — 파이프라인에 들어간 실행마다 1건 생성

필드명	타입	설명
build_id	string	실행 식별자
started_at / finished_at	datetime	실행 시작·종료 시각
schema_version	string	Manifest 형식 버전 (1.0.0)
status · partial	string · bool	ok | failed | cancelled. partial은 취소된 실행에서만 true
inputs / outputs	tuple<string>	소스 출력 키 / 생성된 산출물 경로
warnings / errors	tuple<string>	경고 / 실패 메시지
failures	tuple<RunFailure>	실패 내역: source_key, stage, code, summary
row_counts	dict<str,int>	소스·조인별 Silver 행 수
schema_summaries	dict	소스별 스키마 요약
provenance	tuple<SourceProvenance>	수집 시각, 파라미터, 레코드 수, 체크섬
quality_results	dict	평가한 품질 검사 결과 (pass 포함)
schema_drift · drift_evaluation	dict	이전 실행 대비 변화, 축별 평가 여부
gold_selection	dict	Silver 입력 행 수, Gold 출력 행 수, 적용 규칙
pii_masking	dict	Gold별 마스킹한 컬럼과 예외 컬럼
composition	CompositionProvenance?	조인 내역
warehouse_failures	dict	커밋에 실패한 테이블과 사유
artifacts	dict	Gold 디렉터리별 산출물 다이제스트와 작성기
build_environment	BuildEnvironment?	Python·kpubdata·Builder 버전
inputs_fingerprint (+ algorithm)	string?	입력 데이터 지문 (sha256:…)과 산출 방식
split_algorithm	string?	비율 분할 알고리즘 (hash-sort-v2)
reproducibility · checkpoints_not_reused	dict	체크포인트에서 이어받은 소스, 처음부터 다시 받은 소스
created_by · owner_id · retry_of	string?	요청자 표시 이름, 소유자(응답에서 제외), 재시도 대상 실행

- ※ 출처: manifest/models.py

테이블 정의서 - SQLite 표

○ 주요 표와 컬럼

표	컬럼	설명
builds	run_id (PK), status, started_at, finished_at, spec_digest, error, created_by, dataset_id, owner_id, error_code, error_stage	실행 인덱스. status는 ok·failed·cancelled만 허용. finished_at·dataset_id 인덱스
build_events	seq, run_id, timestamp, event, status, source_key, stage, message, metrics	실행 이벤트 타임라인
run_submissions	run_id (PK), owner_id, created_by, submitted_at	비동기 제출 기록
run_endings	run_id (PK), status, stage, code, summary, recorded_at	Manifest 없이 끝난 실행의 종료 기록
tables	id (PK), workspace_id, logical_name, current_snapshot_id, revision	웨어하우스 테이블. (workspace_id, logical_name) 유일
table_snapshots	id (PK), table_id (FK), run_id, schema_version, coverage_hash, artifact_digest, row_count, state, created_at, committed_at, owner_id	스냅샷. state: staging·validated·committed·quarantined·abandoned·retiring
analyses	analysis_id (PK), workspace_id, owner_id, name, sql, row_limit, table_name, snapshot_id, hold_id, result_meta, created_at	저장된 분석. 읽은 스냅샷을 함께 보관

- ※ 출처: `store/build_index.py`, `events/store.py`, `warehouse/catalog.py`, `service/analyses_api.py`

테이블 정의서 - SQLite 저장소 목록

파일	표	용도
`_builds.sqlite`	builds	실행 인덱스 (Manifest에서 재구성 가능)
`_build_events.sqlite`	build_events, run_submissions, run_endings	실행 이벤트, 제출·종료 기록
`_warehouse.sqlite`	tables, table_snapshots, snapshot_leases, snapshot_holds	웨어하우스 카탈로그와 스냅샷 보존
`_publish_receipts.sqlite`	publish_receipts, publish_receipt_audit	게시 영수증과 감사 기록
`.service/analyses.sqlite3`	analyses	저장된 분석
`.service/revisions.sqlite3`	revisions, revision_audit	명세·주석 개정 이력
`.service/uploads.sqlite3`	uploads	업로드 메타데이터 (큰 본문은 파일로 보관)
`.service/provider-credentials.sqlite3`	provider_credentials	제공 기관 자격 증명 (암호화)
`.service/provider_tests.sqlite3`	provider_tests	연결 확인 결과
`.service/users.sqlite3`	users	가입 신청 원장

- 실행 작업공간 `{output}/{run_id}/`에는 manifest.json, buildspec.yaml, bronze/, silver/, gold/를 파일로 보관
- 웨어하우스 스냅샷은 `{warehouse}/tables/{id}/snapshots/{id}/`에 Parquet으로 보관
- 질의 결과 내보내기 묶음은 `.service/exports/`에 보관
- ※ 출처: `store/build_index.py`, `events/store.py`, `warehouse/catalog.py`, `store/inventory.py`

테이블 정의서 - 명세 검증 오류 코드

○ validate_spec이 반환하는 문제의 형식: code, path, message, hint

검사 영역	주요 오류 코드
필수 항목	empty_field, missing_sources
소스 식별	empty_field, blank_alias
내보내기	unsupported_export_kind
메타데이터	empty_metadata_key
분할	missing_ratios, empty_ratio_name, ratio_sum_mismatch, empty_split_key, unsupported_split_mode, time_column_random_split
스키마 계약	unknown_dtype, unknown_cast_dtype, unknown_read_as_type 외 결측 토큰·병합·자리 채움·이름 변경·파생 규칙
소스 종류	unsupported_source_kind, invalid_upload_id, unsupported_source_format, unknown_encoding, unsupported_source_method, unsafe_url_scheme, url_userinfo_forbidden
소스 키	unsafe_alias, unsafe_source_key, duplicate_source_key, source_key_path_collision
파라미터 축	param_grid_not_supported, empty_param_grid_axis, param_grid_shadows_params, invalid_param_grid_value
개인정보 정책	pii_allow_with_publish
라이선스	missing_license_for_publish, license_other_needs_terms, license_terms_without_other
품질	empty_range_rule, inverted_range_rule, unknown_severity_column
조인	duplicate_source_alias, self_join, unknown_composition_source, duplicate_join_key_column, composition_name_collision
Gold 선택	gold_selection_with_composition

- 검사는 실행 자원을 만들기 전에 수행하며 문제를 모두 모아 한 번에 반환
- 조인 키의 존재와 자료형 호환은 실행 중에 확인 (검증 단계에서는 데이터를 읽지 않음)
- ※ 출처: `spec/validator.py`

핵심 소스코드(1) — kpubdata-builder · run_build()

```
def run_build(spec: BuildSpec, *, client: SourceClient, output_root: Path,
              run_id: str | None = None, retry_of: str | None = None,
              catalog: TableCatalog | None = None, ...) -> BuildResult:
    validate_spec(spec)                                  # 위반 시 ValidationError, 기록 없음
    context = BuildContext.create(spec, output_root=output_root, run_id=run_id)
    ...
    # Per-source fetch/stage mostly waits on network I/O, so concurrent
    # execution via thread pool reduces total time (#247). executor.map
    # returns results in spec.sources order, not completion order, so
    # downstream merge (manifest) stays deterministic.
    max_workers = min(len(spec.sources), _MAX_PARALLEL_SOURCES)
    clear_temp_root(context.output_root / context.run_id)
    try:
        with ThreadPoolExecutor(max_workers=max_workers) as executor:
            results = list(executor.map(_worker, spec.sources))
    ...
    if cancelled:
        status = "cancelled"
    elif errors:
        status = "failed"
        recorder.run_failed(failed_source_count=len(errors))
    else:
        status = "ok"
        recorder.run_finished()
    ...                                                  # ok일 때 웨어하우스 커밋, manifest.json 기
록
```

- BuildSpec을 검증한 뒤 실행 작업공간을 만들고, 소스별 Bronze→Silver→Gold 파이프라인을 병렬로 실행해 상태를 판정하는 핵심 로직
- 결과는 명세 순서대로 모아 Manifest가 실행마다 같은 순서로 기록되게 함
- `...`은 생략한 부분. 원본: `src/kpubdata_builder/pipeline/orchestrator.py`

핵심 소스코드(2) — kpubdata-builder · validate_spec()

```
def validate_spec(spec: BuildSpec) -> None:
    """Validate BuildSpec minimum executability.

    Raises:
        ValidationError: When one or more validation rules fail.
    """
    problems: list[ValidationProblem] = []
    if not spec.dataset_id.strip():
        problems.append(_p("empty_field", "dataset_id", "dataset_id must be a non-empty
string"))
    if not spec.title.strip():
        problems.append(_p("empty_field", "title", "title must be a non-empty string"))
    if not spec.description.strip():
        problems.append(_p("empty_field", "description", "description must be a non-empty
string"))
    if not spec.sources:
        problems.append(_p("missing_sources", "sources", "at least one source is required"))
    ...
    # exports may be empty (#703). A build with none ends at a committed table,
    # which is a complete job — publishing is an explicit follow-up, not the only
    # way to finish.
    ...
    problems.extend(_split_problems(spec))
    problems.extend(_schema_problems(spec))
    problems.extend(_source_kind_problems(spec))
    problems.extend(_source_key_problems(spec))
    problems.extend(_param_grid_problems(spec))
    problems.extend(_pii_problems(spec))
    problems.extend(_license_problems(spec))
    problems.extend(_quality_problems(spec))
    problems.extend(_composition_problems(spec))
    problems.extend(_gold_selection_problems(spec))
    if problems:
        raise ValidationError([str(p) for p in problems], structured=problems)
```

- 실행 자원을 만들기 전에 명세의 문제를 모두 모아 한 번에 알려 주는 사전 검증 로직
- 반환값은 없으며, 문제가 있으면 code·path·message·hint를 담은 `ValidationError` 발생
- `exports`는 비어 있어도 되며, 이 경우 테이블 커밋까지만 수행
- `...`은 생략한 부분. 원본: `src/kpubdata_builder/spec/validator.py`

핵심 소스코드(3) — kpubdata-studio · 근거 대조

```
export function crossCheckAssistantResponse(
  response: AssistantStructuredResponse,
  evidence: AssistantEvidence,
  knownRefs: AssistantKnownRefs,
): CrossCheckResult {
  const rejectedRefs: string[] = [];
  const evidenceRefs = response.evidenceRefs.filter((ref) => {
    const known = isKnownEvidenceRef(ref, knownRefs, evidence);
    if (!known) rejectedRefs.push(`${ref.kind}:${ref.id} (${ref.label})`);
    return known;
  });

  const rejectedActions: string[] = [];
  const suggestedActions = response.suggestedActions.filter((action) => {
    const check = isKnownAction(action, knownRefs, evidence);
    if (!check.ok) rejectedActions.push(check.reason);
    return check.ok;
  });

  let generatedSql = response.generatedSql;
  let rejectedSqlReason: string | undefined;
  if (generatedSql) {
    if (evidence.context.stage !== generatedSql.stage) {
      rejectedSqlReason = t("sqlStageMismatch", { ... });
      generatedSql = null;
    } else if (generatedSql.source && !knownRefs.sourceKeys.has(generatedSql.source)) {
      ...                                                    // 소스가 하나면 소스 지정만 제거, 여럿이면 SQL
폐기
    }
  }
  ...
}
```

- LLM 답변이 인용한 근거·제안 동작·SQL을 Builder에서 받은 실제 근거와 대조하는 로직
- 근거에 없는 항목은 그 항목만 버리고 나머지 답변은 유지
- SQL은 현재 단계·소스와 맞지 않으면 버리며, 실행 전 최종 검사는 Builder가 수행
- `...`은 생략한 부분. 원본: `src/features/assistant/crossCheck.ts`

핵심 소스코드(4) — kpubdata-builder · 읽기 전용 SQL 검사

```
def validate_read_only_sql(sql: str) -> ValidatedSql:
    ...
    expression = statements[0]
    if not isinstance(expression, exp.Query):
        raise UnsafeQueryError("only SELECT queries are allowed")

    for with_node in expression.find_all(exp.With):
        if with_node.args.get("recursive"):
            raise UnsafeQueryError("recursive CTEs are not allowed")
        for cte in with_node.expressions:
            if cte.alias.casefold() == "dataset":
                raise UnsafeQueryError("CTE alias must not shadow dataset")

    _reject_unsupported_relations(cast(exp.Expression, expression))
    _reject_unsafe_functions(cast(exp.Expression, expression))
    root_scope = build_scope(expression)
    if root_scope is None:
        raise UnsafeQueryError("query scope could not be validated")

    for scope in root_scope.traverse():
        _validate_scope(scope)
    if _reachable_dataset_refs(root_scope) == 0:
        raise UnsafeQueryError("query must reference the logical dataset table")

    canonical = expression.copy()
    for node in canonical.walk():
        node.comments = []
    return ValidatedSql(canonical.sql(dialect="duckdb", pretty=False))
```

- 사용자와 AI 도우미가 보낸 SQL을 실행 전에 검사하는 로직
- SELECT 한 문장만 받고, 허용하지 않는 관계·함수를 거부하며, 논리 테이블 `dataset`을 참조하는지 확인
- 통과한 문장은 주석을 뺀 표준형으로 다시 만들어 반환하고, 실행에는 이 표준형만 사용
- `...`은 생략한 부분. 원본: `src/kpubdata_builder/query/security.py`

테스트 설계

대상	종류	규모·기준
Builder	단위 테스트 (pytest)	테스트 파일 271개
	통합·파이프라인 테스트	통합 7개, 파이프라인 6개, 엔진 동등성 1개
	정적 검사	ruff (lint·format), mypy (src 전체)
	실행 환경	Python 3.10·3.11·3.12·3.13 각각 실행, 문서 빌드(mkdocs --strict)
Studio	단위 테스트 (Vitest)	테스트 파일 308개. *.test.ts는 node, *.test.tsx는 jsdom 환경
	커버리지 기준	구문 84%, 분기 75%, 함수 85%, 줄 86% 이상
	E2E (Playwright)	스펙 26개. 데스크톱·모바일(Pixel 7)·CSP(운영 빌드) 프로젝트
	화면 기준 이미지	5개 화면 × 3종(데스크톱, 390px, 다크) = 15장. 차이 200픽셀 이내
	접근성	axe-core로 화면별 검사
연동	계약 호환	Builder 계약과 Studio 스키마의 불일치 검사, kpubdata main과의 통합 테스트 (매일)
	실연동	실제 Builder·다중 사용자·Keycloak 대상 E2E는 별도 설정으로 실행

- 화면 기준 이미지는 고정된 컨테이너에서 만들어 글꼴과 브라우저 차이를 없앰
- Studio의 기능별 클라이언트는 Builder용과 데모용이 같은 계약 테스트를 통과해야 함

CI·배포 구성

GitHub Actions

워크플로	내용	실행 시점
`ci.yml`	lint, 형 검사, 테스트, 문서 빌드	PR, main 푸시
`security.yml`	의존성 점검, 비밀값 검사, CodeQL	PR, 푸시, 매주
`docker.yml`	서비스 이미지 빌드·검증·게시	PR, 푸시, 버전 태그
`cross-repo-contract.yml`	kpubdata main과의 통합 테스트	매일
`image-rescan.yml`	게시한 이미지 재검사	매일
`dataset-automation.yml`	신규 데이터셋 탐색, 활용 신청 승인 확인	매일
`scheduled-*.yml`	분야별 정기 수집·게시	분야별 주기
`release.yml`	릴리스 준비와 태그	수동

배포

- Builder: Docker 이미지로 HTTP 서비스 실행. 산출물과 상태는 `/data` 볼륨에 보관
- 인증 없이 띄울 수 없음 (API 키 또는 OIDC 필수). 개발 모드에서만 인증 생략 가능
- 설정은 환경변수로 주입: 인증·접근, 제공 기관 키, 실행·동시성, 업로드·URL 소스, 상태 저장·웨어하우스, 게시
- 기동할 때 설정을 검사하고, `settings` 명령으로 실제 적용 값을 확인
- 제공 기관 자격 증명은 마스터 키로 암호화해 저장
- Studio: 정적 빌드를 nginx 이미지로 제공. Builder 주소와 OIDC 설정은 컨테이너를 띄울 때 주입 (`/config.js`)
- 데모 모드: Builder 없이 고정 데이터로 동작하며 모든 화면에 Demo 배지 표시

참고

- KPubData Builder 기준 코드: 0539bca (2026-10-12, API 계약 1.123.0)
- KPubData Studio 기준 코드: 08e3e8a (2026-10-12, 버전 0.4.0)
- 화면 그림은 Studio를 데모 모드로 실행해 캡처한 것으로, 표시된 값은 데모 데이터임
- 기술 설명별 코드 근거: 근거 목록
- 2026. 09. 08 판 대비 변경 사항: 제작설계서 변경 내역